



Informatics 2A: Tutorials and Labs (2016)

Links to tutorial and lab sheets will become live by the Friday before the week of the tutorial or lab in question. Links to tutorial solutions will become live after the week of the tutorial.

Week	Tutorial or Lab	Solutions
3	Tutorial 1	Tutorial 1 solutions
3	Lab 1: Introduction to Python/NLTK I	Lab 1: Solutions
4	Tutorial 2	Tutorial 2 solutions
4	Lab 2: Introduction to Python/NLTK II	Lab 2: Solutions
5	Tutorial 3	Tutorial 3: Solutions
5	Lab 3: Support for Assignment 1	
6	Tutorial 4	Tutorial 4: Solutions
6	Lab 4: Support for Assignment 1	
7	Tutorial 5	Tutorial 5: Solutions
8	Tutorial 6	Tutorial 6: Solutions
9	Tutorial 7	Tutorial 7: Solutions
10	Tutorial 8	Tutorial 8: Solutions
10	Lab 5: Support for Assignment 2	
11	Tutorial 9	Tutorial 9: Solutions
11	Lab 6: Support for Assignment 2	

[Home](#) : [Teaching](#) : [Courses](#) : [Inf2a](#)

Informatics Forum, 10 Crichton Street, Edinburgh, EH8 9AB, Scotland, UK
 Tel: +44 131 651 5661, Fax: +44 131 651 1426, E-mail: school-office@inf.ed.ac.uk
 Please [contact our webadmin](#) with any comments or corrections. [Logging and Cookies](#)
 Unless explicitly stated otherwise, all material is copyright © The University of Edinburgh

Informatics 2A 2016–17

Tutorial Sheet 1 (Week 3)

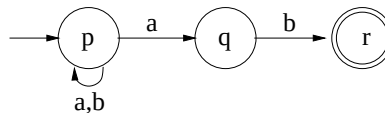
JOHN LONGLEY

This week's tutorial is devoted to the 'pure theory' of regular languages. The relevant lectures are numbers 3, 4 and 5. Next week's tutorial will cover some applications of this theory.

Please attempt the first five questions in advance of tutorials. Whilst all of these cover material it is important to understand, you should especially prioritize questions 1, 3 and 5. Don't spend too long on any one question. If you get stuck, go on to the next.

An optional Question 6 is included for those who like tennis.

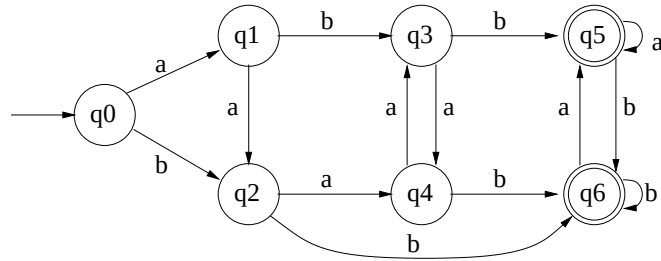
1. Use the subset construction from Lecture 3 to convert the following NFA into an equivalent DFA. You need only include in your DFA those states that are reachable from the initial state.



2. Give both an NFA and a regular expression for each of the following languages. Try to keep the number of states in your NFAs as small as possible.
 - (a) (Over $\{a, b\}$.) The set of strings in which a and b *alternate*: that is, strings not containing aa or bb consecutively.
 - (b) (Over $\{a, b\}$.) The set of strings that do contain aa or bb consecutively. (This is the *complement* of the language in part (a).)
 - (c) (Over $\{a, b\}$.) The set of strings that contain the consecutive sequence $abba$.
 - (d) (Over $\{a, b, c\}$.) The set of strings in which between any two occurrences of a , there must be at least one occurrence of b .

Construct NFAs (possibly with ϵ -transitions) corresponding to the following regular expressions. The construction of your NFAs should broadly follow the structure of the regular expressions as indicated in the lectures, though you may omit ϵ -transitions that you can see to be superfluous.

- (e) $(a^* + b^*)^*$
 - (f) $((aa)^*bb + ab)^*$
 - (g) $\emptyset^*$
3. Use the algorithm from Lecture 4 to minimize the DFA below.



4. (Adapted from Kozen.) Convince yourself intuitively that the following identities are valid for regular expressions. Recall that an equality $\alpha = \beta$ between regular expressions means that the equality of languages $\mathcal{L}(\alpha) = \mathcal{L}(\beta)$ holds.

$$(\alpha + \beta)\gamma = \alpha\gamma + \beta\gamma$$

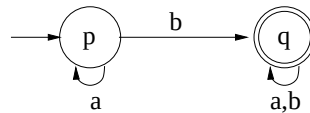
$$\alpha(\beta + \gamma) = \alpha\beta + \alpha\gamma$$

$$(\alpha\beta)^*\alpha = \alpha(\beta\alpha)^*$$

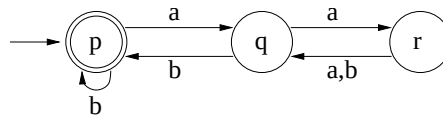
Consider the regular languages over $\{0, 1, 2\}$ denoted by the following pairs of regular expressions. In each case, say whether the two languages are equal or not. If they are, use the above laws (plus any other laws from Lecture 5 that you may require) to prove the equivalence. If they are not, give an example of a string that is in one language but not the other.

- (a) $(0 + 1)^*$ and $0^* + 1^*$
 - (b) $0(120)^*12$ and $01(201)^*2$
 - (c) $(0^*1^*)^*$ and $(0^*1)^*$
 - (d) $(01 + 0)^*0$ and $0(10 + 0)^*$
5. Convert each of the following DFAs into a regular expression, by using Arden's rule to solve a system of simultaneous equations as indicated in Lecture 5.

(a)



(b)



6. (Optional.) In this question we'll consider a finite-state model for an ordinary game of singles tennis (not a tiebreak). If you're needing to brush up on the scoring system for tennis, try this:

https://en.wikipedia.org/wiki/Tennis_scoring_system

The players are Federer and Murray, and Federer is serving. We'll model a game simply as a string over the alphabet $\{f, m\}$, where f means 'point to Federer' and m means 'point to Murray'.

- (a) Construct a DFA that accepts precisely those strings in $\{f, m\}^*$ that correspond to complete games won by Murray. Your DFA should have a state for each possible scoreline that can occur: e.g. 30-15; Deuce.
- (b) Is your DFA *minimal*? If not, which states may be identified? (Do this part just by inspection—I don't recommend applying the algorithm from lectures to your DFA!)
- (c) Now consider an entire tennis match (with up to five sets, and including tiebreaks as necessary) as a sequence of points in this way. Would it be possible to construct a DFA in this style that accepts precisely those sequences that correspond to a complete *match* won by Murray?

Inf2a: Lab 1

Introduction to Python

This short overview of Python. A more comprehensive tutorial for Python 2.6 can be found online at:

<http://docs.python.org/2.6/tutorial/>

That tutorial offered by Python.org contains more on control statements, functions and their definitions, data structures, modules, classes and the Python library.

1 Interpreter

In this session we will launch the Python interpreter and directly execute commands in the terminal window. Python is an interpreted language, so one of its great advantages is the ability to launch an interpreter in this way. This can save a lot of time when you are developing software: when you are not sure about something, you can often test it directly from the interpreter, without the overhead of having a small program to launch from command line. The interpreter is launched by typing the following command in the shell:

```
python
```

This launches the default version of Python on DICE (version 2.6). Once python has been launched, lines previously inserted in the interpreter shell can be recalled by pressing the $\uparrow$ cursor button. (The $\leftarrow$ and $\rightarrow$ cursor buttons also work as expected to move backwards and forwards through the characters in the current line.) To exit the interpreter, type **<CTRL>+D**

In Python, **formatting has meaning** - this means the spacing is very important. Groups of statements are indented under a header. Blocks of code are nested by increasing the indentation. There are no end-of-line symbols (like the semicolon; in Java or C). Instead, newline marks the end of a line. Below we will introduce you to the basic types.

★ In this lab, type the lines introduced by `>>>` and by `...`. Cutting and pasting whole sections will disrupt the formatting and will not work.

1.1 Basic Types

1.1.1 Numbers

Python can be used as a simple calculator. Expression syntax is straightforward: the operators `+`, `-`, `*` and `/` work just like in most other languages.

★ Enter the following expressions in the Python interpreter you have just launched:

```
>>> 2+2
>>> 3*2
>>> 2 + 7.3
```

1.1.2 Strings

Strings can be enclosed in single or double quotes:

```
>>> 'hello'
>>> 'how\'s it going?'
>>> "how's it going?"
>>> 'This is "strange"'
```

If a string is enclosed in a pair of triple quotes, either `"""` or `'''`, it retains whatever formatting is found between them — for example:

```
>>> """
... This is a string that when
... printed maintains its format
... and its end of lines.
... """
>>> print """
... This is a string that when
... printed maintains its format
... and its end of lines.
... """
```

Strings can be concatenated using the `+` operator, and can be repeated with the operator `*`:

```
>>> 'Hello ' + ', ' + 'how are you'
>>> 'help' + '!'*5
```

The individual characters of a string can be accessed using indices. For example the first character has index 0. Substrings can be specified with slice notation, two indices separated by colon. When using slices, indices can be thought of as pointing **between** characters, instead of pointing to characters: the left edge of the first character is 0 and so on.

The following commands illustrate this with the help of a user-defined variable called `word`. Take care when you name your variables not to use reserved words. For a list of these, look here:

http://docs.python.org/2/reference/lexical_analysis.html#keywords.

```

>>> word="hello"
>>> word[0]
>>> word[2]
>>> word[0:2]
>>> word[2:5]
>>> word[:2]
>>> word[2:]

```

Negative indices start counting from the right end.

```

>>> word[-1]
>>> word[-2:]

```

Strings cannot be modified once they are created. Trying to modify a substring results in an error. However, it is easy to create a new string by concatenating substrings from other strings.

1.1.3 Lists

There are different types of compound structures in Python. The most versatile is the list.

```

>>> L = ['monday', 'tuesday', 'wednesday', 'thursday', 'friday']
>>> L

```

Like strings, list items can also be accessed using indices. Similarly, they can be sliced and concatenated:

```

>>> L[0]
>>> L[3]
>>> L[1:]
>>> L[:2]
>>> L[1:3]
>>> L + ['saturday', 'sunday']

```

One can verify the membership of an element to a list using the keyword `in`.

```

>>> 'wednesday' in L
>>> 'sunday' in L

```

Items can be added at the end of a list using the `append(item)` method of the `list` object.

```

>>> L3 = []
>>> L3.append(1)
>>> L3.append(2)
>>> L3

```

It is possible to measure the length of a list using the built-in function `len(list)`.

```

>>> len(L)

```

It is also possible to create nested lists:

```
>>> L1 = [1, 2, 3]
>>> L2 = ['one', L1, 'two']
>>> L2
```

◦ `insert(i,x)` inserts the item `x` at position `i`.

```
>>> L2 = ['a', 'b', 'd', 'e']
>>> L2.insert(1, 'c')
>>> L2
```

◦ `remove(x)` removes the first item in the list whose value is `x`.

```
>>> L2.remove('d')
>>> L2
```

◦ `pop()` returns (and removes) the last item in the list; likewise, `pop(i)` returns and removes the item in position `i`.

```
>>> L2.pop()
>>> L2
```

◦ `sort()` sorts the items of the list, in place.

```
>>> L2.sort()
>>> L2
```

◦ `reverse()` reverses the elements of the list, in place.

```
>>> L2.reverse()
>>> L2
```

◦ `del` can be used to remove items from a list using the index. It can also be used to remove slices from a list.

```
>>> del L2[1:3]
>>> L2
```

You can iterate over a list retrieving the index and the value at the same time using the `enumerate(list)` function.

```
>>> for i, v in enumerate(['a', 'b', 'c', 'd']):
...     print i, v
...
```

1.1.4 Boolean Types

The Boolean type is named `bool`; it takes any Python value and converts it to `True` or `False`.

```
>>> bool(1)
>>> bool(0)
>>> bool([1])
>>> bool([])
```


1.1.5 Tuples

A *tuple* is composed by a number of values separated by commas, and enclosed by parentheses.

```
>>> T = (1,2,'three')
>>> T
>>> T[2]
```

Tuples can be nested.

```
>>> T1 = (1,2,(3,5))
>>> T1
```

Tuples, like strings, are immutable and cannot be changed once created. If you try, you will get an error message.

```
>>> s = "hello"
>>> s[1] = "u"
>>> T1[2] = 3
```

1.1.6 Dictionaries

Dictionaries are very useful structures. They are indexed by **keys**, unlike lists which are indexed by indices. The keys can be any immutable objects (numbers, strings, tuples - you cannot use lists, which are mutable).

A dictionary can be seen an unordered set of *key:value* pairs, with the constraint that keys need to be unique. The main operations performed on dictionaries are storing and retrieving values by their keys.

```
>>> num = {'one':1, 'two':2, 'three':3, 'four':4}
>>> num['three']
>>> num
```

You can easily add a new item to the dictionary:

```
>>> num['five'] = 5
>>> num
```

You can delete one item from the dictionary using the built in function `del(item)`:

```
>>> del(num['three'])
>>> num
```

To list all the keys from a dictionary, you use the method `keys()`. To check if a key belongs to the dictionary you use the method `has_key()`:

```
>>> num.has_key('one')
>>> num.keys()
```

You can iterate over a dictionary, retrieving the keys and their corresponding values using the method `iteritems()`:

```
>>> for k, v in num.iteritems():
...     print k,v
...
```

1.2 Modules

Programs can become long, and it is a good approach to divide them into more manageable units. In Python, programs can be divided into **modules**. The modules can be imported directly into the interpreter or into other programs. Python has a wide library of predefined functions and classes that can be imported and used.

```
>>> import math
```

To call a function imported in a module, it must be prefixed with the module name, to avoid name conflicts. (N.B. The function `math.ceil(x)` below returns as a float, the smallest integer greater than or equal to x.)

```
>>> math.pi
>>> math.ceil(2.35)
```

We can also import only the functions we need. If we do this, it is not necessary to prefix them with the module name.

```
>>> from math import ceil
>>> ceil(2.35)
```

`dir(modulename)` will return a list of names defined in the module. If you modify a module, you need to reload it using `reload(modulename)`.

1.2.1 Pattern Matching

The `re` module provides tools for regular expressions.

```
>>> import re
```

◦ `match(pattern, string)` if zero or more characters at the beginning of `string` match the regular expression `pattern`, it returns a corresponding `MatchObject` instance. It returns `None` if the string does not match the pattern. For a guide to writing regular expressions in machine syntax, see the slides for Informatics 2A Lecture 6.

```
>>> re.match("(aa|bb)+", "aabbbaa")
>>> re.match("(aa|bb)+", "abba")
```

◦ `findall(pattern, string)` returns a list of all non-overlapping matches of `pattern` in `string`.

```
>>> re.findall("[a-z]*th[a-z]*", "I think this is the right one")
```

◦ `sub(pattern, repl, string)` returns the string obtained by replacing the leftmost non-overlapping matches of `pattern` in `string` by the replacement `repl`.

```
>>> re.sub("[a-z]*th[a-z]*", "TH-word", "I think this is the right one" )
```

★ End your interpreter session using <CTRL>+D.

2 Text Editors

In the rest of this lab we will use the types explained above. You may find it useful to refer back.

From here on we will use Python scripts rather than using the interpreter directly. Launch your preferred editor (emacs, vim, gedit,...) in the background (using `&`). If you need further advice on using an editor try an emacs tutorial such as the one within emacs itself: type "C-h t" (control-h, t) within emacs.

Remember: in Python, formatting has meaning - this means the spacing is very important. Groups of statements are indented under a header. Blocks of code are nested by increasing the indentation. There are no end-of-line symbols (like the semicolon; in Java or C). Instead, newline marks the end of a line.

★ Before launching Python, open a terminal window and create a new directory called "MyPython":

```
mkdir MyPython
```

★ Go to this directory:

```
cd MyPython
```

2.1 Control Structures

Python offers the usual control flow statements, as do other languages like Java or C. The `for` loop is more powerful than in most of the other languages.

At this point you might want to write the commands as a program in an editor (such as emacs, vim or gedit), so that you can easily modify the lines in case of errors. You can then execute the program from the shell.

For example, to open the emacs editor, type the following command from the shell:

```
emacs program_name.py &
```

Then write your program in the editor. To save the program, press `<CTRL>+X` and then `<CTRL>+C`. This will prompt you to save the file. Press 'y'. This will take you back to shell.

To launch your program, type the following command from the shell:

```
python program_name.py
```

Now we will consider the python control structures `if`, `while` and `for`.

2.1.1 If Statement

★ Using an editor, create a file `binary.py` containing the following lines:

```
x=raw_input("Enter a binary sequence : ")
if (x[0] == '0'):
    print "Starts with 0"
elif (x[0] == '1'):
    print "Starts with 1"
else:
    print "Error: Not a binary number!"
```

★ After saving the file <CTRL>+X and then <CTRL>+S, launch it from the shell.

```
python binary.py
```

★ When asked, insert a binary sequence (such as 00101 or 10010)

It is possible to create more complex conditions using the keyword `and` for conjunction and the keyword `or` for disjunction – e.g.

```
if (x[0] == '0' or x[0] == '1'):
    print "Starts with " + x[0]
else:
    print "Error: Not a binary number!"
```

2.1.2 While

★ Using the editor create a file `downtoone.py`. Type the following lines.

```
x = 10
while (x > 0):
    print x
    x = x - 1
```

★ Save the file and then launch the program from the shell.

2.1.3 For Loop

The `for` statement iterates over the items of any sequence (such as strings or lists), using the keyword `in` discussed earlier.

★ Using an editor, create a file `printlist.py`. Type the following lines.

```
L = ['monday', 'tuesday', 'wednesday', 'thursday', 'friday']
for x in L:
    print x
```

★ Save the file and then launch the program from the shell.

To iterate over a sequence of numbers, the built-in function `range()` can be used to automatically generate the sequence:

★ Re-enter the python interpreter and type in the following lines.

```
>>> range(5)
>>> range(5,10)
```

★ Using an editor, create a file `printnumbers.py`. Type the following lines.

```
for x in range(10):
    print x
```

★ Save the file and then launch the program from the shell.

You can write an `if` statement nested inside a `for` statement as follows, which prints the even numbers less than 10. Remember that nesting is indicated by increased indentation.

★ Using an editor, create a file `printevennumbers.py` and enter the following lines, where `%` is the “modulo” operator.

```
for x in range(10):
    if (x%2 == 0):
        print x
```

★ Save the file and then launch the program from the shell.

2.2 Functions

A function is introduced by the keyword `def`, followed by the function name and by its list of parameters in parentheses. The statements that form the function body start on the next line, and are indented. The first line can optionally be a string that describes the function. This string can be used by automatic generators of documentation.

You can either type the code in the interpreter or alternatively type it as a program and execute it from shell.

```
>>> def square(value):
...     """Returns the square value of the value"""
...     return value*value
...
>>> square(4)
```

3 Exercises

EXERCISE 1

Create a regular expression that checks if a string starts with 3 binary digits (and test it: `010asda` must be recognised, while `1aa` must be rejected)

Using a regular expression, write a python statement that finds all the words that end with “ly” in strings (and test it, for example using the sentence “it is likely to happen rarely”)

Using a regular expression, write a python statement that replaces all the words that start with “wh” by “WH-word” (and test it, for example in the sentence “who should do what?”)

EXERCISE 2

Write a program that, for every element in the list
`['how', 'why', 'however', 'where', 'never']`
prints:

- a star symbol ‘*’
- the first two letters from the element (its 2-character prefix)
- the whole element

i.e. producing something like:

```
* ho how
* wh why
...
```

EXERCISE 3

Modify the previous program to print a star (*) in front of the elements that start with the prefix “wh” and a hyphen ‘-’ in front of the others, producing something like:

```
- ho how
* wh why
...
```

EXERCISE 4

Write a function `checkPrefix(list, prefix)` in the python interpreter. This function should wrap the loop created in Exercise 2. When the function is called, it should print the contents of `list`, adding a star in front of those elements that start with the two-character prefix in `prefix`. Check what happens when `checkPrefix` is given an empty `list`.

EXERCISE 5

Using an editor, create a new file and type in the function `checkPrefix(list,prefix)`. Save the file as “`checker.py`” in your “`MyPython`” directory. Then import your module in the python interpreter and call the function.

Informatics 2A 2016–17

Tutorial Sheet 2 (Week 4)

JOHN LONGLEY

This week's exercises concern some applications of regular languages and finite state machines (Lectures 6 and 7), along with the problem of determining whether a language is regular or not (Lecture 8).

Recall that there are two common notations for regular expressions, both of which are described in the lecture slides:

- The basic *mathematical notation*, as introduced in Lecture 5, and used e.g. in Kozen's book.
- A more extended *machine syntax*, described in Lecture 6, and used by the Grep tools and in languages such as Perl and Python.

For example, the language written in mathematical notation as $a^*(b+c)^* + dd^*$ could be written in machine syntax as `(a*[bc]*)|d+`. Questions 1 and 2 involve the use of both notations. Unfortunately, there are clashes between them: e.g. they differ on the meanings of both '+' and '.'.

Please attempt the following three questions in advance of your tutorials next week. You should be sure to attempt at least part of Question 3, since the pumping lemma is a topic that often causes some difficulty to begin with. The last (starred) subquestion of Question 3 is an optional challenge question, and need only be attempted by those confident with the material.

1. A *floating-point literal* in Java (simplifying slightly) has the following form: a sequence of decimal digits, then a decimal point, then another sequence of decimal digits. *At least one* of these digit sequences must be non-empty, so `.` by itself is not allowed. The whole of this may then *optionally* be followed by an *exponent part*, consisting of the symbol `E` or `e`, then an optional `+` or `-` sign, then a sequence of one or more decimal digits. E.g.,

2. .3 3.14 6.0e23 7.E-1

- (a) Write a regular expression in mathematical notation that defines this class of floating-point literals. You may introduce abbreviations for any sets of characters you may find useful.
- (b) Write an `egrep` command that finds all substrings of a source file `foo.java` that conform to the above format for floating-point literals. You may find it helpful to try out your command on the DICE machines, using a small test file containing some examples and non-examples of floating-point literals.
- (c) Suppose we have an NFA N for the class of floating-point literals (you need not construct one). How many non-accepting states may N pass through between two successive occurrences of accepting states?

(Note in passing that when `egrep` finds nested matches to a pattern it returns only the longest supermatches. For example, if `foo.java` contains the literal `0.0`, then it will also contain `'0.'` and `'0.'`, but only `'0.0'` will be returned.)

2. In Java, a *string literal* is a sequence of *input characters* enclosed in double quotes, e.g. "Catch22". The only restrictions on the sequence of characters are that the double quote character " itself, if it appears in the body of the string, must be denoted by the *escape sequence* \", and the character \ itself must be denoted \\. Five other escape sequences denoting control characters are also allowed: \b, \t, \n, \f, \r.
 - (a) Write a regular expression in mathematical notation that defines the set of Java string literals. Again, you may introduce abbreviations for any sets of characters you find useful.
 - (b) Write an **egrep** command that will print all lines of the file `foo.java` containing a valid string literal. Here you should bear in mind that **egrep** patterns have their own escape conventions: the character " must be written within a pattern as \", while \ must be written as \\ and moreover must be enclosed within square brackets []. Again, you may find it helpful to test your command on the DICE machines.
 - (c) Ignoring escape characters and comments, a line of Java code will typically be erroneous if it contains an *odd* number of occurrences of ". Write a **grep** command that prints all lines from `foo.java` that have this property.
3. Each of the following expressions defines a language over $\Sigma = \{a, b\}$. If the language is regular, give an NFA or regular expression that defines it. If it is not regular, use the pumping lemma to prove this.
 - (a) The set of *palindromes*, i.e. those strings $x \in \Sigma^*$ such that $x = \text{rev}(x)$, where $\text{rev}(x)$ means x reversed (written backwards).
 - (b) The set of strings of the form ss , i.e. those with two identical halves. (E.g. *abbabb*.)
 - (c) The set of strings with an equal number of substrings ab and ba .
 - (d)* The set of strings whose length is a prime number, i.e.,

$$\{x \in \Sigma^* \mid |x| \text{ is prime}\} .$$

Inf2a: Lab 2

Object-Oriented Programming and NLTK

1 Object-Oriented Programming

A class is defined using the keyword `class`, followed by the class name. The class body must be indented, and the first statement can be a documentation string.

A method is a function that “belongs to” a class. Methods must have, as their first parameter, the variable `self`, that contains the reference to the object itself (for example `def greet(self, name)`: It is equivalent to the variable `this` in Java, with the difference that you need to define it explicitly.

By default, all methods and attributes in Python are public. It is possible to make them ‘pseudo’ private, adding two underlines before their name (for example as in `__func(self)`).

★ Type the lines introduced by `>>>` and by `...`

Remember that indentation matters in Python, which means that in the following example, you cannot simply hit `<Enter>` on the third line; you will need to type the appropriate number of spaces/tab first.

```
>>> class MyClass:
...     """A very simple class"""
...
...     def greet(self, name):
...         return "Hello, " + name
... 
```

A class is instantiated into an object, and methods of the object are called like in Java.

★ Type the lines introduced by `>>>` and by `...`

```
>>> c = MyClass()
>>> c.greet("Paolo")
```

Classes can have an initialisation method `__init__()` similar to the class constructor in Java. This method is called when the class is instantiated and can have a set of parameters. In contrast with Java, that can have many different

constructors, there can be a only one such methods per class.

★ Type:

```
>>> class Greeter:
...     """A simple class"""
...     def __init__ (self, greeting):
...         self.greeting = greeting
...     def greet(self, name):
...         return self.greeting + ", " + name
...
>>> c2 = Greeter("hi")
>>> c2.greet("tim")
```

A class can derive from another class:

★ Type:

```
>>> class GreeterEx(Greeter):
...     """A derived class"""
...     def bye(self):
...         return "Bye Bye"
...
>>> c3 = GreeterEx("hello")
>>> c3.greet("mr smith")
>>> c3.bye()
```

This class will contain the methods defined in **Greeter**, plus the new **bye()** method.

2 Passing Parameters

It is possible to pass parameters to a script.

★ Create in your editor a file named **test.py**

★ Type in the editor:

```
import sys
for arg in sys.argv:
    print arg
```

★ Save the file

★ Type in the shell:

```
python test.py these are the arguments
```

The arguments are stored in the variable **sys.argv**, that is a list of string. **sys.argv[0]** contains the name of the script, while the following elements contains the arguments.

3 Natural Language Toolkit (NLTK)

NLTK is a suite of Python libraries and programs for symbolic and statistical natural language processing. It has extensive documentation, including tutorials that explain the concepts underlying the language processing tasks supported by the toolkit (<http://nltk.org/>).

★ Open a terminal window, and go to the folder “MyPython” that you created during Lab 1.

★ Launch the python shell using the command `python`.

To load the NLTK libraries we use the `import` statement.

★ Type:

```
>>> from nltk.tokenize import simple
```

A description of a class is available using the `help` function.

★ Type:

```
>>> help(simple)
```

Type `q` to finish the help mode. You can check the documentation of the NLTK API to see all available modules and classes in:

<http://nltk.org/api/nltk.html>

4 Tokens

For most kinds of linguistic processing, we need to identify and categorise the words of the text. This turns out to be a non-trivial task. Here, we introduce *tokens* as the building blocks of text, and show how texts can be tokenized.

★ Type:

```
>>> from nltk.tokenize import WhitespaceTokenizer
>>> text = 'Hello world! This is a test string.'
>>> WhitespaceTokenizer().tokenize(text)
```

As you can see, tokenization based on whitespace alone is not sufficient. The method `tokenize.regexp` uses a regular expression to determine how text should be split up. This regular expression specifies the characters that can be included in a valid word.

★ Type:

```
>>> from nltk.tokenize import RegexpTokenizer
>>> text = "Hello. Isn't this fun?"
>>> pattern = r'\w+|[\^\w\s]+'
>>> tokenizer = RegexpTokenizer(pattern)
>>> tokenizer.tokenize(text)
```

The statement:

```
|r'\w+|[\^\w\s]+'
```

creates a regular expression which accepts sequences formed by “word” characters, i.e., characters other than whitespace or punctuation (the subexpression `\w+`) or by characters which are neither word characters nor whitespace characters (`[\^\w\s]+`, the sign `^` means complement). The previous regular expression is not good enough with the string `$22.40` and `44.50%`, where we might want to keep the symbol `$` and `%` attached to the number.

★ Type:

```
>>> text = 'That poster costs $22.40.'
>>> tokenizer.tokenize(text)
```

The union of the previous regular expression with this one: `\$\d+\.\d`, where `\d` represents a decimal digit, solves the first problem.

★ Type:

```
>>> pattern = r'\w+|\$\d+\.\d|[\^\w\s]+'
>>> tokenizer = RegexpTokenizer(pattern)
>>> tokenizer.tokenize(text)
```

A note on the union operator (`|`):

`A|B`, where `A` and `B` are both regular expressions, creates a regular expression that will match either `A` or `B`. An arbitrary number of regular expressions can be separated by the `|` in this way. As the target string is scanned, regular expressions separated by `|` are tried from left to right. When one pattern completely matches, that branch is accepted. This means that once `A` matches, `B` will not be tested further, even if it would produce a longer overall match. In other words, the `|` operator is never greedy. To match a literal `|`, escape it using `\`, or enclose it inside a character class, as in `[|]`.

For more information about how to create regular expressions check:

<http://http://docs.python.org/2/library/re.html#re-syntax>.

5 Corpora

NLTK is distributed with several corpora. They can be accessed using the `corpus` package.

First we import the Brown Corpus, the first million word, part-of-speech tagged electronic corpus of English. Each of the sections *a* through *r* represents a different genre. List of available sections can be accessed using the `items` and `documents` variables.

```
>>> from nltk.corpus import brown
>>> brown.categories()

>>> brown.fileids()

>>> help(brown)
```

★ Type:

The first argument of the `sents` method is the fileid of the corpus. Try the example with `'ca02'`.

★ Type:

With the `tagged_sents` method we can access the tagged text of a corpus. Every word in the sentence is tagged and the sentence is presented as a list of binary tuples containing the word and its tag.

```
>>> print brown.tagged_sents('cn01')[0]
```

★ Type:

```
>>> dict = {}
>>> for fileid in brown.fileids():
...     for sntc in brown.tagged_sents(fileid):
...         for word,tag in sntc:
...             if tag in dict:
...                 dict[tag] += 1
...             else:
...                 dict[tag]=0
... 
```

```
>>> for tag,count in dict.items():
...     print tag," ",count
...
```

We can use a `ConditionalFreqDist` to find the most frequent occurrence of a word in a context, for instance the previous word. First, we pass each token of the corpus together with its previous token to the `ConditionalFreqDist` class. We call this process *training*.

★ Type:

```
>>> from nltk.probability import ConditionalFreqDist
>>> cfdist = ConditionalFreqDist()

>>> prev=None
>>> for fileid in brown.fileids():
...     for sntc in brown.sents(fileid):
...         for token in sntc:
...             cfdist[prev][token] += 1
...             prev=token
...
```

Notice that the first token of the corpus has no “previous word”. For this example, use `None` as the context for the first token.

We can see the words which follow another word using the `samples()` method.

★ Type:

```
>>> cfdist['living']
```

Try this with a different word of your own choice.

We can use the information stored in `ConditionalFreqDist` to create a text generator using the most frequent word given a word. The following code creates a sentence of word length 20 starting with 'an'. The most frequent word is given by `max()`.

Please note that this will generate strings which may not be well-formed sentences.

★ Type:

```
>>> word = 'an'
>>> for i in range(20):
...     print word,
...     word = cfdist[word].max()
...
```

Try this with a different word.

6 Grammars

The `grammar` module defines a set of classes that can be used to define context free grammars:

- The `grammar.Nonterminal` class is used to represent nonterminals.
- The `grammar.Production` class is used to represent (CFG) productions.
- The `grammar.ContextFreeGrammar` class is used to represent CFGs.

`Nonterminal` is a simple class that is used to let NLTK distinguish terminals from nonterminals.

★ Type:

```
>>> from nltk import grammar as cfg
>>> S = cfg.Nonterminal('S')
>>> S

>>> NP = cfg.Nonterminal('NP')
>>> NP

>>> VP, Adj, V, N = cfg.nonterminals('VP, Adj, V, N')
>>> VP, Adj, V, N
```

Each `Production` specifies that a nonterminal (the left-hand side of a rule) can be expanded to the sequence of terminals and nonterminals given in the right-hand side of the rule.

★ Type:

```
>>> prod1 = cfg.Production(S, [NP, VP])
>>> prod1

>>> prod2 = cfg.Production(NP, ['the', Adj, N])
>>> prod2
```

Context free grammars are encoded by the `CFG` class. A CFG consists of a special start nonterminal, and an ordered list of productions.

★ Type:

```
>>> grammar = cfg.CFG(S, [prod1, prod2])
>>> grammar

>>> grammar.start()

>>> grammar.productions()
```

Additionally, with the `fromstring` function of the `CFG` class, it is possible to create a grammar from its text description.

★ Type:

```
>>> from nltk import CFG
>>> grammar2 = CFG.fromstring('''
...     S -> NP VP
...     NP -> "I" | "John" | "Mary" | "Bob" | Det N
...     VP -> V NP | V NP PP
...     V -> "saw" | "ate"
...     Det -> "a" | "an" | "the" | "my"
...     N -> "dog" | "telescope" | "apple"
...     PP -> P NP
...     P -> "on" | "with"
...     ''')
```

```
>>> grammar2
```

```
>>> grammar2.start()
```

```
>>> grammar2.productions()
```

For a grammar, we can parse a sentence and get its syntactic tree using the `RecursiveDescentParser()` function.

★ Type:

```
>>> from nltk import parse
>>> from nltk.tokenize import WhitespaceTokenizer
>>> from nltk.parse import RecursiveDescentParser
>>> sntc1 = WhitespaceTokenizer().tokenize('I saw Mary')
>>> sntc2 = WhitespaceTokenizer().tokenize('John ate my apple')
>>> rd_parser = RecursiveDescentParser(grammar2)

>>> for p in rd_parser.parse(sntc1):
...     print p

>>> for p in rd_parser.parse(sntc2):
...     print p
```

7 Treebank

NLTK also includes a 10% fragment of the Wall Street Journal section of the Penn Treebank. Each sentence of the corpus is available in three forms; (1) as tokenized text, (2) as tokens labelled with part of speech, and (3) as parse trees. These can be accessed using `treebank.raw()` for the raw text, `treebank.tagged()` for the tagged text, and `treebank.parsed()` for the parse trees.

★ Type:

```
>>> from nltk.corpus import treebank
>>> help(treebank)
```

```

>>> print treebank.raw('wsj_0001.mrg')

>>> print treebank.words('wsj_0001.mrg')

>>> print treebank.tagged_words('wsj_0001.mrg')

>>> print treebank.parsed_sents('wsj_0001.mrg')[0]

```

The argument in the above three functions, 'wsj_0001', is a section of the treebank. We can see the list of all sections using `treebank.items`. The function `treebank.parsed()` [] returns an object of class `nltk.Tree`. We can analyse the tree using different functions available in this class.

★ Type:

```

>>> t = treebank.parsed_sents('wsj_0001.mrg')[0]
>>> t.label()

>>> t.leaves()

>>> print t

>>> len(t)

>>> t[0]

>>> t[0].label()

>>> len(t[0])

```

`t.label()` contains the top-most node in the tree `t`. We can see the list of words that yields the parse tree using `t.leaves()`. Using `len(t)`, we can get the number of child nodes to `t.label()`. Each of these child nodes and their subtrees can be accessed using `t[0].label()` and `t[0]` respectively.

Using the above utilities, we can find out the subject of any given sentence. The following code prints the subject NP phrase in the first sentence of 'wsj_0001'

★ Type:

```

>>> t = treebank.parsed_sents('wsj_0001.mrg')[0]
>>> for ch_tree in t:
...     if (ch_tree.label().startswith('NP-SBJ')):
...         print ch_tree.leaves()

```

8 Exercises

EXERCISE 1

Create a class named **Queue** that models a queue: the first element that enters is the first that exits (FIFO: First In, First Out). The class will use a list to maintain the data. It will expose the following methods:

- **isempty()**: verifies if the queue is empty
- **push(item)** inserts an element at the end of the queue
- **pop()**: extracts and returns the first element in the queue (possibly only if the queue is not empty)

Import the module into the python shell, and test it Remember to create the list that contains the data before accessing to it.

EXERCISE 2

Using your preferred editor, create a class named **Stack** that models a stack: the last element that enters is the first that exits (LIFO: Last in, First Out). The class will use a list to maintain the data. It will expose the following methods:

- **isempty()**: verifies if the stack is empty
- **push(item)**: inserts an element at the end of the stack
- **pop()**: extracts and returns the last element of the stack (possibly only if the stack is not empty)

Import the module into the python shell, and test it. Remember to create the list that contains the data before accessing to it.

EXERCISE 3

Using your favourite editor, create a file that defines a class **Checker** that contains the function you wrote in Exercise 5 of Lab 1. Save the file as “**oo_checker.py**” in your “**MyPython**” directory. Then import your new module from the python shell, instantiate the class into an object and call the method.

EXERCISE 4

Using your preferred editor, create a class that checks if a string (**infix**) is contained in other strings. The class must be initialised passing the **infix** string, that must be stored in a class variable. The class must expose the method **check(string)** that verifies if **infix** is contained in the passed string (you can use the operator **in** to verify if a string is contained in another one: **string1 in string2**).

Hint Remember to use **self** when trying to access to class methods and attributes.

Import your module into the python shell, and test its behaviour (you must instantiate the class passing the infix string, and then call the method check passing different strings)

Hint If you use the statement `import modulename`, remember to use the modulename prefix in front of the class name. If you make an error in the class, and you need to reimport the module, use `reload(modulename)`. `import` will not reimport a module already imported. You will also have to reinstantiate the class.

EXERCISE 5

Create a class for managing a phone book. The user must be able to:

- insert a name and the relevant phone number,
- obtain a number from a name,
- verify if name is in the book,
- list all the names and phone numbers in the book,
- delete a name from the book
- as optional feature, the user should be able to print in alphabetical order the names and their phone numbers

Import your class into the python shell, and test it (remember to instantiate the class).

Hint Use a dictionary to store the data, and remember to create the it before using it. You can use the method `keys()` to obtain the list of all the keys. Then you can apply any method available for the lists on the list you have obtained.

EXERCISE 6

Add to the pattern:

```
r'\w+|\$\d+\.\d+|[\^w\s]+'
```

the option to tokenize percentages as a single tokens, for instance:

- 100.00%, 10.5%, and 10.234%.

Hint Use the union operator (`|`) to add your option to the start of `pattern`. Test with the following text:

```
>>> text = 'The interest does not exceed 8.25%.'
```

EXERCISE 7

Use the following code as a starting point:

```
>>> from nltk.corpus import brown
>>> from nltk.probability import ConditionalFreqDist
>>> cfdist = ConditionalFreqDist()

>>> prev=None
>>> for fileid in brown.fileids():
...     for sentc in brown.sents(fileid):
...         for token in sentc:
...             cfdist[prev][token] += 1
...             prev=token
...

>>> cfdist['living']

>>> word = 'an'
>>> for i in range(20):
...     print word,
...     word = cfdist[word].max()
...
```

With 'an', the text generator gets stuck in a cycle within the first 20 words. Modify the program to choose the next word randomly from a list of the most likely words in the given context. (Hint: store the most likely words in a list `lwords`, and then randomly choose a word from the list using `random.choice(lwords)`. Don't forget to `import` the `random` module. In order to find the most likely words, use `sorted()` instead of `samples()`).

- Remember that we saw at the start of this section that each part of the Brown Corpus corresponds to a different genre of English. For example, Part *j* corresponds to scientific text. Select a genre of your choice from the Brown Corpus. Train your system on this corpus and get it to generate random text. You may have to experiment with different start words.
- Try the same approach with a different genre.
- Compare the generated texts. How do the resulting texts differ?.

EXERCISE 8

Create a new grammar (`grammar3`) using as a base `grammar2` (from section 6) plus the production `NP -> Det N PP`.

`grammar2`:

```
S -> NP VP
NP -> "I" | "John" | "Mary" | "Bob" | Det N
VP -> V NP | V NP PP
V -> "saw" | "ate"
```

```

Det -> "a" | "an" | "the" | "my"
N -> "dog" | "telescope" | "apple"
PP -> P NP
P -> "on" | "with"

```

- Parse the sentence *Mary saw the dog with the telescope*. How many parse trees do you get?
- For `grammar2`, write down two unambiguous sentences (that yields just one tree), two ambiguous sentences (that yields more than one tree), and two ungrammatical sentences (that yields no tree at all).

EXERCISE 9

```

>>> t = treebank.parsed_sents('wsj_0001.mrg')[0]
>>> for ch_tree in t:
...     if (ch_tree.label().startswith('NP-SBJ')):
...         print ch_tree.leaves()

```

- Extend the above program to identify the subject in all the sentences in 'wsj_0003'.
- A subordinate clause in a sentence will have its own subject. So, extend the code (using recursion) to identify all the subjects in every sentence.

Informatics 2A 2016–17

Tutorial Sheet 3 (Week 5)

JOHN LONGLEY

This week's exercises cover the basics of context-free grammars, pushdown automata, LL(1) grammars and predictive parsing (Lectures 8–11).

Question 4 (on constructing parse tables) represents perhaps the most technically demanding topic of the entire course. You should make sure you are comfortable with the ideas in Question 3 before attempting Question 4.

Throughout this sheet, we use n to stand for some lexical class of *numeric literals*, such as 5 or 23. The precise definition of this class is not relevant to these exercises.

1. In this question, ideas from the course turn and bite their own tail. We consider a context-free grammar for the (mathematical) language of regular expressions!

Fix some alphabet Σ . The terminals of our grammar will be the symbols

$$\text{sym} \quad \emptyset \quad \epsilon \quad + \quad * \quad (\quad)$$

where sym denotes the lexical class consisting of symbols from Σ . (Note that ϵ is an actual symbol here, in contrast to ϵ which denotes the empty string.) There are two nonterminals: RegExp (the start symbol) and Atom . The productions are as follows:

$$\begin{aligned} \text{RegExp} &\rightarrow \text{Atom} \mid \text{RegExp} + \text{RegExp} \mid \text{RegExp RegExp} \\ &\quad \mid \text{RegExp} * \mid (\text{RegExp}) \\ \text{Atom} &\rightarrow \text{sym} \mid \emptyset \mid \epsilon \end{aligned}$$

Note that this grammar represents (as is often done in practice) the concatenation of regular languages by juxtaposition of regular expressions, instead of using an explicit infix ‘.’ operation.

In Lecture 5, we gave the following mathematical definition of the language $\mathcal{L}(e)$ associated with a regular expression e .

- $\mathcal{L}(\emptyset) = \emptyset$, $\mathcal{L}(\epsilon) = \{\epsilon\}$, $\mathcal{L}(a) = \{a\}$ ($a \in \Sigma$)
- $\mathcal{L}(\alpha + \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$
- $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \cdot \mathcal{L}(\beta)$
- $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$

Strictly speaking, what $\mathcal{L}$ operates on here is not just a regular expression e in textual form, but a particular *syntax tree* for e : we need to know how e is parsed in order to make sense of $\mathcal{L}(e)$.

- (a) Give an example of a ‘harmless’ ambiguity arising from the above grammar; i.e., an ambiguous string for which alternative syntax trees all give rise to the same regular language. Draw two different syntax trees for this string.

- (b) Give an example of a ‘harmful’ ambiguity; i.e., a string for which different syntax trees describe different regular languages. Draw two syntax trees for the string that do indeed describe different regular languages.
2. An NPDA $(Q, \Sigma, \Gamma, s, \Delta)$ (as defined in the Lecture 9 slides) is said to be *real-time deterministic* if it has no ϵ -transitions, and for every $q \in Q$, $a \in \Sigma$, $x \in \Gamma$ there is *at most one* pair (q', β) with $((q, a, x), (q', \beta)) \in \Delta$.¹ When working with deterministic PDAs, it is usual to assume the input alphabet Σ includes a special end-of-input symbol $\$$ which is required to occur at the end of every input string (and nowhere else).
- Provide real-time deterministic PDAs that accept the following languages, using acceptance by empty stack.
- (a) The language $\{a^n b^n \mid n \geq 0\}$. (Hint: use two control states.)
- (b) The set of well-matched bracket sequences involving two kinds of brackets: $(\dots)$ and $[\dots]$. For example, $[([] ())]$ is a well-matched sequence, while $([])$ is not.
3. Consider the following grammar for generating arithmetic expressions such as $(n * n * n)$.

Terminals: $(,), *, n$
 Nonterminals: Exp, Ops
 Productions: $\text{Exp} \rightarrow n \text{Ops} \mid (\text{Exp})$
 $\text{Ops} \rightarrow \epsilon \mid * n \text{Ops}$
 Start symbol: Exp

Convince yourself that this is in fact an LL(1) grammar, and that its parse table is

	$($	$)$	$*$	n	$\$$
Exp	(Exp)			$n \text{Ops}$	
Ops		ϵ	$* n \text{Ops}$		ϵ

(Here, for example, the top left entry (Exp) stands for the production $\text{Exp} \rightarrow (\text{Exp})$.)

- (a) Using this table, apply the LL(1) parsing algorithm to the input

$(n * n)$

At each step, show the operation applied, the input string remaining, and the stack state, as in Lecture 11, Slide 6.

- (b) For each of the following three input strings, explain how and where an error arises in the course of the LL(1) parsing algorithm. In each case, suggest an error message that an LL(1) parser could reasonably issue to the author of the input string.

$() \quad n) \quad n *$

¹The reason for the adjective “real-time” is that the standard notion of *deterministic PDA* is more general in that ϵ -transitions are permitted in certain circumstances, see, e.g., Chapter F of Kozen.

- (c) The grammar in this question is unnaturally restrictive. For example, it does not admit the string $(n * n) * n$. Design an LL(1) grammar that recognises the full language of arithmetic expressions such as $(n * n) * (n * n * (n * n))$; i.e., the language of arithmetic expressions involving just $*$ and n , but allowing unrestricted (well-matched) bracketing.
4. Consider the following grammar for generating boolean conditions such as $n + -n * n == n$.

Terminals:	$n, +, *, -, ==$
Nonterminals:	Cond, Exp, TimesExp, OptMinus, TimesOps, PlusOps
Productions:	Cond $\rightarrow$ Exp == Exp Exp $\rightarrow$ TimesExp PlusOps TimesExp $\rightarrow$ OptMinus n TimesOps OptMinus $\rightarrow \epsilon \mid -$ TimesOps $\rightarrow \epsilon \mid * n$ TimesOps PlusOps $\rightarrow \epsilon \mid +$ TimesExp PlusOps
Start symbol:	Cond

- (a) Identify the set of nonterminals from which the empty string can be derived.
- (b) Calculate the *First* sets for each of the nonterminals in the grammar.
- (c) Calculate the *Follow* sets for each of the nonterminals in the grammar.
- (d) Using this information, try to build a *parse table* for the grammar. Is the grammar LL(1) or not?

Informatics 2A 2016–17

Tutorial Sheet 4 (Week 6)

JOHN LONGLEY

This week's exercises concern material from Lectures 12 and 13 on formal languages, and Lecture 14 on morphology parsing for natural languages.

1. Recall the grammar for mathematical regular expressions from last week's sheet:

$$\begin{aligned}\text{RegExp} &\rightarrow \text{Atom} \mid \text{RegExp} + \text{RegExp} \mid \text{RegExp} \text{ RegExp} \\ &\quad \mid \text{RegExp} * \mid (\text{RegExp}) \\ \text{Atom} &\rightarrow \text{sym} \mid \emptyset \mid \epsilon\end{aligned}$$

Using methods from Lecture 13, construct an LL(1) grammar for the language of regular expressions that is equivalent to the grammar given earlier. Your grammar should embody the usual precedence conventions for regular expressions: $*$ takes precedence over concatenation, which takes precedence over $+$. (If you are in doubt as to whether your grammar is LL(1), see whether you can construct a parse table for it.)

2. The *concrete syntax* for Micro-Haskell (MH) types (see Lecture 12 and Assignment 1) is specified by the LL(1)-grammar:

$$\begin{aligned}\text{Type} &\rightarrow \text{Type1 TypeOps} \\ \text{TypeOps} &\rightarrow \epsilon \mid \rightarrow \text{Type} \\ \text{Type1} &\rightarrow \text{Integer} \mid \text{Bool} \mid (\text{Type})\end{aligned}$$

with start symbol `Type`. The *abstract syntax* is specified by the grammar

$$\text{Type} \rightarrow \text{Integer} \mid \text{Bool} \mid \text{Type} \rightarrow \text{Type}$$

- (a) Write out the concrete parse tree for type expression:

$$\text{Integer} \rightarrow \text{Bool} \rightarrow \text{Integer}$$

- (b) Write out the abstract syntax tree that the above parse tree will be converted to.

Consider the following grammar for an abstract syntax of arithmetic expressions (involving only the binary operations $-$ and $+$).

$$\text{Exp} \rightarrow n \mid \text{Exp} + \text{Exp} \mid \text{Exp} - \text{Exp}$$

where (as in the last tutorial) n stands for some lexical class of *numeric literals*.

- (c) Give an LL(1)-grammar for a corresponding concrete syntax of bracketed arithmetic expressions.

- (d) Write out the concrete parse tree for the arithmetic expression:

$$10 - 5 - 4$$

- (e) Write out the abstract syntax tree that the above parse tree should be converted to.
- (f) What difference arises in the process of translating concrete parse trees to abstract syntax trees between the case of MH types and that of arithmetic expressions? What is the origin of this difference?

3. In written English, the *E-deletion* rule states that when a suffix beginning with *e* or *i* is added to a stem ending in *e*, the (first) *e* is deleted. Examples: *love[^]ing* $\Rightarrow$ *loving*, *queue[^]ed* $\Rightarrow$ *queued*. (There are some exceptions to the rule, e.g. *canoeing*, but don't worry about dealing with these.)

Design a finite-state transducer that applies this rule, in the same style as the transducer for E-insertion in the lecture slides. The inputs to the transducer should be strings over $\{a, \dots, z, ^, \#\}$, where $\#$ denotes a word boundary and $^$ a morpheme boundary. The outputs should consist of words separated by spaces, using the alphabet $\{a, \dots, z, \text{SPC}\}$. For simplicity, you may assume that each word consists of at most two morphemes.

You should take care to work within the framework of non-deterministic transducers as defined in the lectures, in which each transition may be associated with (at most) one input and one output character. If your transducer involves a large number of states, you need only draw a representative sample to illustrate the general pattern.

Informatics 2A: Tutorial Sheet 5 (Week 7)

Part-of-speech tagging

1. Suppose you are given a text corpus of 100,000 tokens involving 10,000 distinct word types. Assume for the purpose of this question that the corpus perfectly obeys Zipf's law. (This wouldn't be possible in practice, since the number of occurrences of each word type must be a whole number.)
 - (a) Estimate how many word types are needed to account for *half* the tokens in the corpus. (Your calculation need not be too precise.)
 - (b) The word *about* is the 60th most common word in the corpus. Estimate how many times it occurs.
2. The following are some of the POS tags from the Penn Treebank tagset, illustrated with sample words.

NN	Noun, singular (<i>cat</i>)
NNS	Noun, plural (<i>cats</i>)
VBD	Verb, past tense (<i>sang</i>)
VBG	Verb, gerund form (<i>singing</i>)
JJ	Adjective (<i>huge</i>)
RB	Adverb (<i>hugely, soon, very</i>)
PRP	Personal pronoun (<i>you</i>)
PRP\$	Possessive pronoun (<i>your</i>)
IN	Preposition (<i>on, up</i>)
DT	Determiner (<i>the, some</i>)
CC	Coordinating conjunction (<i>and, but</i>)
WRB	Wh-adverb (<i>where, why</i>)

Tag the following text, ignoring punctuation. You may assume the above tags are sufficient for this purpose. Highlight any points at which you think difficult or debatable tagging decisions arise.

I was walking down the high street yesterday when I noticed an old man acting suspiciously. He was peering into various shop windows and writing things in a notebook. When he spotted me, he stuffed the notebook into his pocket and wandered off.

(For this question, getting the 'right answer' is less important than having the experience of trying.)

3. Consider the following (artificial) sentence:

The old man the lifeboats

Use the version of *bigram tagging* described in the lectures to tag this sentence, using the tags DT, N, V, Adj and the following frequency data. (Rows correspond to potential POS tags for the word in question; columns

correspond to the POS tag of the preceding word.) You may assume *the* and *lifeboats* can only be tagged as DT and N respectively.

old	DT	N	V	Adj	man	DT	N	V	Adj
N	8	2	3	2	N	102	45	15	86
V	0	0	0	0	V	0	11	4	4
Adj	34	5	13	17	Adj	0	0	0	0

4. Now use the *Viterbi algorithm* to tag the sentence

The old man the lifeboats

using the following transition and emission probabilities. Include explicit backtrace pointers in your Viterbi matrix.

	DT	N	V	Adj		lifeboat	man	old	the
start	0.4	0.3	0.1	0.2	DT	0	0	0	0.5
DT	0	0.6	0	0.4	N	0.2	0.3	0.2	0
N	0.05	0.3	0.4	0.25	V	0	0.1	0	0
V	0.4	0.3	0.1	0.2	Adj	0	0	0.4	0
Adj	0.1	0.5	0.2	0.2					

Transitions

Emissions

You may want to use a calculator to help with the arithmetic. Note too that in the transition matrix, rows represent the ‘previous state’, and columns represent the ‘next state’ (the opposite of the convention in question 3).

Informatics 2A: Tutorial Sheet 6 (Week 8)

CYK and Earley Parsing

ADAM LOPEZ

1. Consider the following context free grammar with start symbol S:

$S \rightarrow NP VP$	$PP \rightarrow Pre NP$
$S \rightarrow I VP PP$	$V \rightarrow ate$
$NP \rightarrow Det N$	$Det \rightarrow the \mid a$
$VP \rightarrow ate NP$	$N \rightarrow fork \mid salad$
$VP \rightarrow V$	$Pre \rightarrow with$

- (a) Convert this grammar to Chomsky Normal Form (see Lecture 12).
 (b) Use the CYK algorithm from Lecture 18 to parse the sentence

I ate the salad with a fork

representing the CYK chart in matrix form.

- (c) How many complete analyses of S do you get? Draw their parse trees.
 (d) How, in general, would you go about translating a parse tree for the CNF grammar into one for the original grammar? Do this for the parse trees obtained from the above sentence.
 (e) Now add a further production rule to your CNF grammar to allow for the alternative prepositional phrase attachment, i.e. ‘the salad with a fork’. Revise your CYK chart or graph to include any new entries this introduces.

How many complete analyses are there now for the above sentence?

2. Consider the following context free grammar with start symbol S:

$S \rightarrow NP VP$	$N \rightarrow saw$
$S \rightarrow Pro V$	$V \rightarrow saw$
$NP \rightarrow Pro$	$Pro \rightarrow I$
$VP \rightarrow V Det N$	$Det \rightarrow the$

- (a) Use the Earley algorithm from Lecture 19 to parse the sentence:

I saw the saw

(Note that you don’t have to convert the grammar to CNF this time.)

- (b) Compare the behaviour of the Earley parser on this sentence to that of CYK. (You need not create the entire CYK chart for this.) How does the latter handle the ambiguity in *saw*?

Informatics 2A: Tutorial Sheet 7 (Week 9)

Probabilistic CFGs and Parameter Estimation

ADAM LOPEZ

1. Consider the following probabilistic context free grammar:

S	→	Subj VP (1.0)
VP	→	V Obj (0.5) V Obj Obj (0.3) V Small (0.2)
Small	→	Obj V (1.0)
Subj	→	I (0.3) NP (0.7)
Obj	→	her (0.2) NP (0.8)
NP	→	N (0.5) Det N (0.5)
V	→	make (0.6) duck (0.4)
N	→	duck (0.5) goose (0.5)
Det	→	her (1.0)

Use a probabilistic CYK-style algorithm to find the most probable parse for the sentence

I make her duck

and determine its probability. (For hints on the meanings of the various possible parses, see the Lecture 19 slides.)

N.B. Do not convert the grammar to Chomsky Normal Form — instead, you should draw up a CYK-*style* parse chart for the grammar as it stands in the manner illustrated in the lectures. For small examples this is perfectly feasible, but you should check that you understand why a CYK-style algorithm has difficulties with non-CNF grammars in general. (Hint: think about what additional inferences you might need to make in order to work with grammars in this form.)

2. (a) Derive a PCFG grammar from the corpus below. Write down the rules of the grammar and calculate their probabilities. Assume for the purpose of the grammar that the corpus is lowercased (e.g. *The* has been replaced by *the*).

```

(S
  (NP She)
  (VP
    (VP
      (V saw)
      (NP (Det the) (N man)))
    (PP
      (P from)
      (NP (Det a) (N distance))))))

(S
  (NP Here)
  (VP

```

```

(V is)
(NP (Det a) (N telescope))))

(NP
  (NP (Det the) (N man))
  (PP (P with) (NP (Det the) (N guitar))))

(S
  (NP He)
  (VP
    (V saw)
    (NP (Det the) (N girl))))

(S
  (NP (Det The) (N man))
  (VP
    (V saw)
    (NP
      (NP (Det the) (N girl))
      (PP
        (P with)
        (NP (Det the) (N flowers))))))

```

- (b) Draw all possible parse trees for the following sentence, and compute their probabilities using the probabilistic grammar you have created.

He saw the man with the telescope

- (c) You can also use your probabilistic grammar for text prediction much like the suggestion box in a search engine. If a user typed **The girl saw**, what is the most likely suggestion for a possible completion?

Informatics 2A: Tutorial Sheet 8 (Week 10)

Agreement; First order logic; Compositional semantics

ADAM LOPEZ

1. As noted in Lecture 23, present tense verbs in English agree with their subject in number (*I sleep; he sleeps; they sleep*). Another agreement constraint in English is that reflexive pronouns (e.g. *herself, itself, themselves*) must agree with the noun phrase they stand for in person, number and gender.

Using the machinery of CFGs with non-terminals and rules parameterized by suitable attributes as in Lecture 22, construct a grammar for a tiny fragment of English that includes three-word sentences containing a transitive verb, such as:

She washes herself
I wash myself
It prepares itself
You prepare yourselves
He congratulates himself
They congratulate themselves

but excludes sentences like

She washes himself
It prepares themselves
We congratulates ourselves

2. Choose a set of constants and predicates suitable for representing the following sentences in first order predicate logic as described in Lecture 24.
 - Jumbo is an elephant.
 - An elephant is a mammal.
 - Every elephant has an owner.
 - Everyone who owns an elephant sings it a song.
 - Only one elephant danced.
 - Every elephant did not dance.

Translate each of these sentences into a formula of FOPL. If you think a sentence is semantically ambiguous, give a first-order formula for each possible interpretation.

3. Consider the following context-free grammar with semantic attachments.

$S \rightarrow NP VP$	$\{ NP.Sem(VP.Sem) \}$	t
$VP \rightarrow IV$	$\{ IV.Sem \}$	$\langle e, t \rangle$
$VP \rightarrow TV NP$	$\{ \lambda x. NP.Sem(TV.Sem(x)) \}$	$\langle e, t \rangle$
$NP \rightarrow Det N$	$\{ Det.Sem(N.Sem) \}$	$\langle \langle e, t \rangle, t \rangle$
$NP \rightarrow John$	$\{ \lambda P.P(John) \}$	$\langle \langle e, t \rangle, t \rangle$
$NP \rightarrow ice-cream$	$\{ \lambda P.P(Ice-cream) \}$	$\langle \langle e, t \rangle, t \rangle$
$Det \rightarrow a \mid an$	$\{ \lambda Q.\lambda P.\exists x Q(x) \wedge P(x) \}$	$\langle \langle e, t \rangle, \langle \langle e, t \rangle, t \rangle \rangle$
$Det \rightarrow every$	$\{ \lambda Q.\lambda P.\forall x Q(x) \Rightarrow P(x) \}$	$\langle \langle e, t \rangle, \langle \langle e, t \rangle, t \rangle \rangle$
$N \rightarrow cat$	$\{ \lambda x.cat(x) \}$	$\langle e, t \rangle$
$N \rightarrow ice-cream$	$\{ \lambda x.ice-cream(x) \}$	$\langle e, t \rangle$
$IV \rightarrow runs$	$\{ \lambda x.run(x) \}$	$\langle e, t \rangle$
$TV \rightarrow likes$	$\{ \lambda x.\lambda y.like(x, y) \}$	$\langle e, \langle e, t \rangle \rangle$

Calculate the semantics of the following phrases and sentences, showing explicitly any β -reduction steps used to simplify λ -expressions.

- John runs
- likes ice-cream
- John likes ice-cream
- an ice-cream
- likes an ice-cream
- John likes an ice-cream
- every cat
- every cat likes ice-cream
- every cat likes an ice-cream

Informatics 2A: Tutorial Sheet 9 (Week 11)

Revision Tutorial

Sample ‘Part A’ exam questions

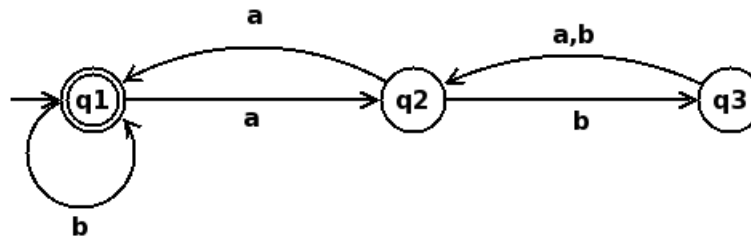
JOHN LONGLEY

The Informatics 2A exam has two parts. Since 2012, Part A consists of five compulsory short questions, worth 10 marks each. Part B consists of three long questions, of which you must attempt two; these are worth 25 marks each. As a guideline, allowing for time taken to familiarize yourself with the paper as a whole, you should aim to spend 10 minutes on each of the Part A questions, and 30 minutes on each Part B question.

This tutorial sheet consists of six sample problems illustrating the style and length of Part A questions. (Naturally, nothing is implied about the selection of topics that will feature in the exam.) The guideline time for completing this tutorial sheet under exam conditions is therefore 1 hour.

Examples of Part B style questions can be found in Informatics 2A exam papers from recent years.

1. (a) Explain briefly what is meant by the *language processing pipeline*. Outline a typical structure for such a pipeline in the case of a programming language such as Java, and in the case of a natural language such as English. [7 marks]
- (b) State one important difference between natural languages and formal ones. Explain, with reference to the language processing pipeline, what difference this might make to how these languages are processed. [3 marks]
2. Consider the DFA below.



- (a) Based on the DFA, write down simultaneous Kleene-algebra equations in three variables X_1 , X_2 and X_3 , representing the languages accepted if the DFA is run starting in states $q1$, $q2$ and $q3$ respectively. [4 marks]
- (b) Use Arden's Rule to solve the simultaneous equations and produce a regular expression for the language accepted by the DFA (whose start state is $q1$). [6 marks]
3. Consider a pushdown automaton with two control states $Q = \{q1, q2\}$, start state $q1$, input alphabet $\Sigma = \{a, b\}$, stack alphabet $\Gamma = \{a, b, \perp\}$,

and transition relation:

$$\begin{array}{lll}
 q1 & \xrightarrow{a, x : ax} & q1 \\
 q1 & \xrightarrow{b, x : bx} & q1 \\
 q1 & \xrightarrow{\epsilon, x : x} & q2 \\
 q2 & \xrightarrow{a, a : \epsilon} & q2 \\
 q2 & \xrightarrow{b, b : \epsilon} & q2 \\
 q2 & \xrightarrow{\epsilon, \perp : \epsilon} & q2
 \end{array}$$

where x can be instantiated with any symbol from the stack alphabet Γ . The automaton accepts on empty stack.

(We use the general notation

$$q \xrightarrow{s, x : \alpha} q'$$

to mean that when the automaton is in control state $q \in Q$ and $x \in \Gamma$ is popped from the top of the stack, the input symbol or empty string $s \in \Sigma \cup \{\epsilon\}$ can be read to reach control state $q' \in Q$ with $\alpha \in \Gamma^*$ pushed onto the stack.)

- (a) Describe in detail an execution of the above PDA that accepts the string

abba

[8 marks]

- (b) Give a concise mathematical definition of the language recognised by the PDA above.

[2 marks]

4. Consider the following probabilistic context-free grammar:

$$\begin{array}{ll}
 S & \rightarrow \text{NP VP} \quad (1.0) \\
 \text{NP} & \rightarrow \text{N} \quad (0.5) \quad | \quad \text{A NP} \quad (0.5) \\
 \text{VP} & \rightarrow \text{V} \quad (0.5) \quad | \quad \text{V NP} \quad (0.5) \\
 \text{N} & \rightarrow \text{fat} \quad (0.2) \quad | \quad \text{ducks} \quad (0.4) \quad | \quad \text{fish} \quad (0.4) \\
 \text{V} & \rightarrow \text{ducks} \quad (0.4) \quad | \quad \text{fish} \quad (0.6) \\
 \text{A} & \rightarrow \text{fat} \quad (1.0)
 \end{array}$$

- (a) Draw up a probabilistic CYK chart (in matrix form) for the sentence

fat ducks fish

Include all entries, whether or not they contribute to some complete parse.

[8 marks]

- (b) Draw the most probable parse tree for the sentence. What is its probability?

[2 marks]

5. Consider the following grammar with associated semantics:

$$\begin{array}{lll}
 S & \rightarrow \text{PN is a NP} & \{ \text{NP.Sem(PN.Sem)} \} \\
 S & \rightarrow \text{PN is not a NP} & \{ \neg \text{NP.Sem(PN.Sem)} \} \\
 \text{NP} & \rightarrow \text{N PrP} & \{ \lambda x. \text{N.Sem}(x) \wedge \text{PrP.Sem}(x) \} \\
 \text{PrP} & \rightarrow \text{Prep PN} & \{ \text{Prep.Sem(PN.Sem)} \} \\
 \text{PN} & \rightarrow \text{London} & \{ \text{London} \} \\
 \text{PN} & \rightarrow \text{Scotland} & \{ \text{Scotland} \} \\
 \text{N} & \rightarrow \text{city} & \{ \lambda z. \text{city}(z) \} \\
 \text{N} & \rightarrow \text{country} & \{ \lambda z. \text{country}(z) \} \\
 \text{Prep} & \rightarrow \text{in} & \{ \lambda y. \lambda z. \text{In}(z, y) \} \\
 \text{Prep} & \rightarrow \text{outside} & \{ \lambda y. \lambda z. \neg \text{In}(z, y) \}
 \end{array}$$

- (a) Compute the semantics of the following sentence. Show your working in detail, including any relevant β -reduction steps.

London is a city outside Scotland

[7 marks]

- (b) Write down the logical formula obtained as the interpretation of

London is not a city in Scotland

You need not show your working in this case.

[1 mark]

- (c) Look at the logical formulae obtained from the two complete sentences above. Does either imply the other via the usual laws of propositional logic? In which direction(s) does the implication hold?

[2 marks]

6. Let Σ be a set of terminals consisting of all words in the English language together with the exclamation mark '!'. We consider a grammar with nonterminals

$S \ E \ A_x$

where x ranges over words in the English language; that is, there is one nonterminal A_x , for every English word x . The grammar contains the following rules.

$$\begin{array}{lll} S & \rightarrow & A_x E \mid A_x S E \\ A_x E & \rightarrow & x ! \\ A_x y & \rightarrow & y A_x \\ y A_x ! & \rightarrow & y y ! A_x \end{array}$$

where x, y again range over English words. The start symbol is S .

- (a) Give a full derivation of

hip hip ! hooray !

[6 marks]

- (b) Give a precise mathematical description of the language L generated by the grammar.

[2 marks]

- (c) State where the language L resides in the Chomsky hierarchy, and give brief justification for why the language does indeed reside at the level you assert. (You do not need to explain why the language does not reside lower down the hierarchy.)

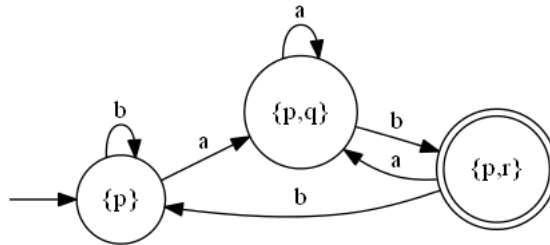
[2 marks]

Informatics 2A 2016–17.

Tutorial Sheet 1 - SOLUTIONS

JOHN LONGLEY

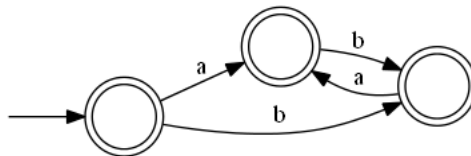
1. Three subsets of $\{p, q, r\}$ suffice:



The best way to produce this is to start with the DFA start state $\{p\}$, and then explore the result of applying a and b transitions to states so far constructed, until no new DFA states (i.e. subsets of $\{p, q, r\}$) arise.

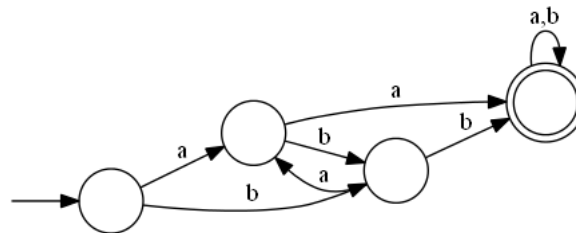
2. The NFAs given here are the 'simplest possible' – however, many other choices of regular expressions would be equally reasonable.

(a) NFA:



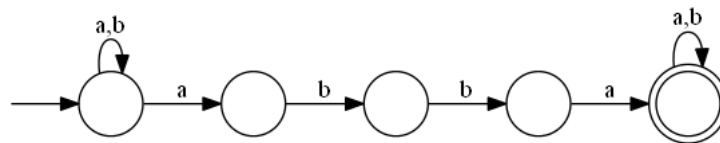
Regular Expression: $(a + \epsilon)(ba)^*(b + \epsilon)$

(b) NFA:



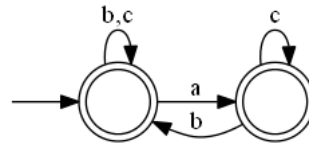
Regular Expression: $(a + b)^*(aa + bb)(a + b)^*$

(c) NFA:



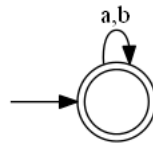
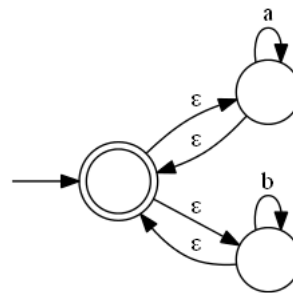
Regular Expression: $(a + b)^*abba(a + b)^*$

(d) NFA:



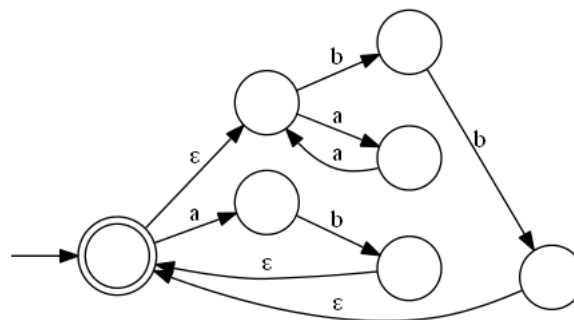
Regular Expression: $\mathcal{Z}(\epsilon + a(\mathcal{Z}b\mathcal{Z}a)^*)\mathcal{Z}$, where $\mathcal{Z} = (b + c)^*$

(e)

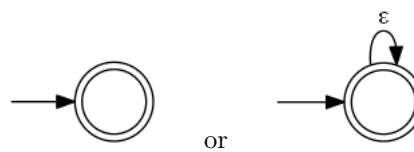


Actually would accept the same language, but I wouldn't regard it as following the structure of the regular expression.

(f)

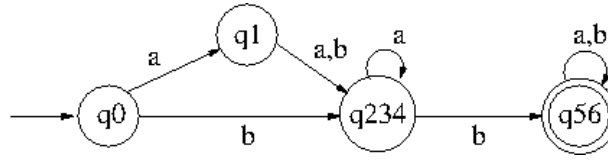


(g)



[Note that ϵ is the only string accepted.]

3. The minimized DFA is:



Please obtain this using the algorithm presented in Lecture 4. However, it can also be obtained more directly by observing the following.

- States $q5$ and $q6$ may be collapsed, since **any** string takes us from either of these to an accepting state.
 - States $q2, q3$ and $q4$ may all be collapsed, since the strings that takes us from these to an accepting state are those matching $a^*b(a+b)^*$.
 - Any other pair of states are differentiated by their behaviour on at least one of the strings: a , ϵ , b , ab .
4. (a) Different: 01 is in $\mathcal{L}((0+1)^*)$ but not $\mathcal{L}(0^*+1^*)$.
- (b) The same: by the third identity with $a = 1$, $b = 20$,
 $(120)^*1 = 1(201)^*$
 where $0(120)^*12 = 01(201)^*2$
- (c) Different: 0 is in $\mathcal{L}((0^*1^*)^*)$ but not $\mathcal{L}((0^*1)^*)$.
- (d) The same: $(01+0)^*0$
 $= (0(1+\epsilon))^*0$ by second identity and ' $a\epsilon = a$ '.
 $= 0((1+\epsilon)0)^*$ by third identity.
 $= 0(10+0)^*$ by first identity and ' $\epsilon a = a$ '.
5. (a) The required language is X_p , where

$$X_p = aX_p + bX_q \quad (1)$$

$$X_q = (a+b)X_q + \epsilon \quad (2)$$

Solving these:

$$\begin{aligned} X_q &= (a+b)^* && \text{from (2) by Arden's rule} \\ X_p &= aX_p + b(a+b)^* && \text{substituting in (1)} \\ X_p &= a^*b(a+b)^* && \text{by Arden's rule.} \end{aligned}$$

(b) The required language is X_p , where

$$X_p = bX_p + aX_q + \epsilon \quad (3)$$

$$X_q = bX_p + aX_r \quad (4)$$

$$X_r = (a+b)X_q \quad (5)$$

Solving these:

$$\begin{aligned}
X_q &= bX_p + a(a+b)X_q && \text{substituting (3) in (2)} \\
X_q &= (a(a+b))^*bX_p && \text{by Arden's rule} \\
X_p &= bX_p + a(a(a+b))^*bX_p + \epsilon && \text{substituting in (1)} \\
&= (b + a(a(a+b))^*b)X_p + \epsilon && \text{by distributivity law} \\
&= (b + a(a(a+b))^*b)^* && \text{by Arden's rule.}
\end{aligned}$$

6. (a) The DFA has 21 states in all (I won't draw it here). There are 16 states corresponding to all possible scorelines x/y where $x, y \in \{0, 15, 30, 40\}$. (Except that the state for 40/40 is known as Deuce.) The start state is 0/0. The transitions between the above states are as expected, e.g. from 15/30 there is an f -transition to 30/30 and an m -transition to 15/40.

There is also state 'Advantage Federer' with an f -transition from Deuce and an m -transition to Deuce. There is a state 'Game Federer' with f -transitions from the states 40/0, 40/15, 40/30 and Advantage Federer. Similarly on Murray's side, except that 'Game Murray' is designated as an accepting state.

Finally, there should also be a 'garbage state' which we enter (and stay in) if input symbols continue after a game has been completed.

- (b) This DFA is not minimal. The main thing to note is that the states 40/30 and Advantage Federer can be identified: from either of these states, f would take us to Game Federer and m would take us to Deuce. Likewise, 30/40 and Advantage Murray can be identified.

Less interestingly, we can identify Game Federer with the garbage state (but this is just a consequence of our biased decision only to accept wins by Murray).

- (c) Yes, an entire tennis match can indeed be modelled using a DFA.

One can build such a DFA in a hierarchical way. First, we can model a set as a sequence of games (say F for Game Federer, M for Game Murray), and build a DFA over $\{F, M\}$ to process complete sets. We then refine this by replacing each state along with its F and M transitions by (essentially) a complete copy of the DFA constructed in (a), or by a suitably adapted version of this if a tiebreak is required. This gives a DFA that processes complete sets at the level of individual points. Repeating the process, we now build a DFA that models a complete match at the level of sets, and then insert lots of copies of the DFA for a single set to obtain a DFA for a complete match at the level of points.

Inf2a: Lab 1 (Solutions)

Introduction to Python

EXERCISE 1

Create a regular expression that checks if a string starts with 3 binary digits (and test it: 010asda must be recognised, while 1aa must be rejected)
`re.match("[0|1][0|1][0|1]+", "010adsa")`

Using a regular expression, write a python statement that ends all the words that end with "ly" in strings (and test it, for example using the sentence "it is likely to happen rarely")
`re.findall("[\w]+ly", "it is likely to happen rarely")`

Using a regular expression, write a python statement that replaces all the words that start with "wh" with "WH-word" (and test it, for example in the sentence "who should do what?")
`re.sub("wh\w+", "WH-word", "who should do what?")`

EXERCISE 2

```
>>> lst = ['how', 'why', 'however', 'where', 'never']
>>> for i in lst:
...     print "*", i[:2], i
```

EXERCISE 3

```
>>> lst = ['how', 'why', 'however', 'where', 'never']
>>> for i in lst:
...     if i[:2]=="wh": print "*", i[:2], i
...     else: print " ", i[:2], i
...
```

EXERCISE 4

```
>>> lst = ['how', 'why', 'however', 'where', 'never']
>>> prefix = "wh"
>>> def checkPrefix(lst, prefix):
...     for i in lst:
...         if i[:2]==prefix: print "*", i[:2], i
```

```
...     else: print " ", i[:2], i
...
>>> checkPrefix(lst,prefix)
```

EXERCISE 5

Copy the code from Ex 4 and save to a file called checker.py

```
>>> lst = ['how', 'why', 'however', 'where', 'never']
>>> import checker
>>> checker.checkPrefix(lst,"wh")
```

Informatics 2A: Tutorial Sheet 2 - SOLUTIONS

JOHN LONGLEY

1. (a) Let $\underline{D}$ (for decimal digits) be the language:

$$0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9.$$

The required regular expression is then:

$$\underline{D}^*(\underline{D} \cdot + \cdot \underline{D})\underline{D}^*(\epsilon + ((E + e)(\epsilon + '+' + '-'))\underline{D} \underline{D}^*))$$

(many variations are possible)

- (b) The `egrep` command is `egrep pattern foo.java`, where *pattern* is:

`"[0-9]*((( [0-9]\. ) | ( \. [0-9] )) [0-9]*([Ee] [+ -]? [0-9]+)?"`

In many versions of the machine syntax, `[0-9]` can be abbreviated e.g. to `[\d]` or `:digits`:

- (c) Two. For example, the inputs `7.` and `7.e-5` are accepted, but `7.e` and `7.e-` are not. (This has implications for longest-match lexing: we cannot simply stop as soon as we pass from an accepting state to a non-accepting one, as we might enter an accepting state again later.)

2. (a) Let $\underline{S}$ be the language of string characters, i.e. all input characters except `"` (double quote) and `\` (backslash).

Then the required regular expression is:

$$"(\underline{S} + \backslash (" + \backslash + \mathbf{b} + \mathbf{t} + \mathbf{n} + \mathbf{f} + \mathbf{r}))^*" "$$

- (b) Translating this into machine syntax with suitable escapes for `"` and `\`, we get:

```
egrep "\"([^\\"\\]|(\\[\\\"\\b\\t\\n\\f\\r]))*\" \" foo.java
```

(Note the two levels of escape sequences involved here.)

- (c) A possible `grep` command is:

```
egrep "^^[^\"]*\"^[^\"]*\\\"^[^\"]*\"^[^\"]*\"*$" foo.java
```

(Note that `^` and `$` match the start and end of line respectively.)

3. In each of (a)-(d) below, let L denote the language in question.

- (a) Not regular.

Given $k \geq 0$, consider $x = \epsilon$, $y = a^k$, $z = ba^k$. Then $xyz \in L$.

Given any splitting of y as uvw where $v \neq \epsilon$, take $i = 0$.

Then $uv^i w = uw = a^l$ for some $l < k$, so $xuv^i wz = a^l ba^k \notin L$.

- (b) Not regular.

Given $k \geq 0$, let $x = a^k b$, $y = a^k$, $z = b$, so that $xyz \in L$.

Given any splitting of y as uvw with $v \neq \epsilon$, we may take $i = 2$ (just for a change).

Then $xuv^i wz = a^k ba^l b$ for some $l > k$, whence $xuv^i wz \notin L$.

- (c) Regular. The trick is to note that the strings of L are exactly those that switch between a 's and b 's an even number of times, i.e. those that start and end with the same letter. This is because we get an ab every time we switch from a 's to b 's, and a ba every time we switch from b 's to a 's.

So L corresponds to the following regular expression:

$$\epsilon + a + b + a(a+b)^*a + b(a+b)^*b .$$

- (d) Not regular.

Given $k \geq 0$, let $p \geq k$ be prime (using Euclid's theorem that there are infinitely many prime numbers).

Consider $x = \epsilon$, $y = a^p$, $z = \epsilon$. Then $xyz = a^p \in L$.

Given any splitting of y as uvw where $v \neq \epsilon$, take $i = p + 1$.

Note that $|xuwz| = p - |v|$ and $|v^i| = (p + 1)|v|$, so $|xuv^i wz| = p - |v| + (p + 1)|v| = p(|v| + 1)$, which is not prime since both factors (p and $|v| + 1$) are ≥ 2 . Thus $xuv^i wz \notin L$.

Note that as i varies, the possible lengths of the string $xuv^i wz$ form an *arithmetic progression*. In the above, we are essentially exploiting the idea that there are no infinite arithmetic progressions of prime numbers.

As a side remark: the Green-Tao Theorem (2004), a celebrated result in number theory, states that, even though the prime numbers do not contain any infinite arithmetic progressions, they do contain arbitrarily long ones. So one can't place any fixed upper bound on the i needed in the above proof in advance.

Inf2a: Lab 2 (Solutions)

Object-Oriented Programming and NLTK

EXCERCISE 1

```
class Queue:
    def __init__(self):
        self.q = []

    def isempty(self):
        return len(self.q)==0

    def push(self,item):
        self.q.append(item)

    def pop(self):
        if not self.isempty():
            return self.q.pop(0)
```

EXCERCISE 2

```
class Stack:
    def __init__(self):
        self.s = []

    def isempty(self):
        return len(self.s)==0

    def push(self,item):
        self.s.append(item)

    def pop(self):
        if not self.isempty():
            return self.s.pop()
```

EXCERCISE 3

```
class Checker:
    def checkPrefix(self, list, prefix):
        for i in list:
            if i[:2]==prefix: print '*', i[:2], i
            else: print ' ', i[:2], i

>>> import oo_checker
>>> c = oo_checker.Checker()
>>> c.checkPrefix(lst,'wh')
```

EXCERCISE 4

```
class Checker:
    def __init__(self,infix):
        self.infix = infix

    def check(self, str):
        return self.infix in str
```

EXCERCISE 5

```
class AddressBook:
    def __init__(self):
        self.b = {}

    def insert(self,name, phone):
        self.b[name]=phone

    def get(self,name):
        return self.b[name]

    def has_name(self,name):
        return self.b.has_key(name)

    def list(self):
        for n,p in self.b.iteritems():
            print n,p

    def delete(self, name):
        del self.b[name]

    def orderedList(self):
        orderedkeys = self.b.keys()
        orderedkeys.sort()
        for n in orderedkeys:
            print n, self.b[n]
```

EXERCISE 6

```
>>> from nltk.tokenize import RegexpTokenizer
>>> text = 'The interest does not exceed 8.25%.'
>>> pattern = r'\d+\.\d+%|\w+|\$+\d+\.\d+|[\^\\w\s]+'
>>> tokenizer = RegexpTokenizer(pattern)
>>> tokenizer.tokenize(text)
['The', 'interest', 'does', 'not', 'exceed', '8.25%', '.']
```

EXERCISE 7

```
>>> from nltk.corpus import brown
>>> from nltk.probability import ConditionalFreqDist
>>> cfdist = ConditionalFreqDist()
>>> prev=None
>>> for fileid in brown.fileids():
...     for sentc in brown.sents(fileid):
...         for token in sentc:
...             cfdist[prev][token] += 1
...             prev=token
...
>>> word='an'
>>> import random
>>> for i in range(20):
...     print word,
...     lwords=cfdist[word].keys()
...     if (len(lwords) > 10):
...         lwords = lwords[:10]
...     word= random.choice(lwords)
...     while word == '.':
...         word = random.choice(lwords)
... 
```

EXERCISE 8

Parse the sentence Mary saw the dog with the telescope. How many parse trees do you get?

2

For grammar3, write down 2 unambiguous sentences (just one tree), 2 ambiguous sentences (more than one tree), and 2 ungrammatical sentences (no tree at all).

unambiguous: “Mary saw the dog”” and “Mary ate the apple””

ambiguous: “I saw the dog on the telescope”” and “I saw an apple with the telescope””

ungrammatical: "I see the dog with the telescope" and "Mary saw on the telescope the apple"

EXCERCISE 9

Extend the program to identify the subject in all the sentences in "wsj_0003":

```
>>> for t in treebank.parsed_sents('wsj_0003.mrg'):
...     for ch_tree in t:
...         if (ch_tree.label().startswith('NP-SBJ')):
...             print ch_tree.leaves()
```

Extend the code to identify all the subjects in a given sentence. A subordinate clause in a sentence will have its own subject. Using recursion, print all the subjects in the sentence:

```
>>> def printsub(t):
...     if (t.node.startswith('NP-SBJ')):
...         print t.leaves()
...     else:
...         if (t.height() > 2):
...             for i in t:
...                 printsub(i)
>>> c = 1
>>> for t in treebank.parsed_sents('wsj_0003.mrg'):
...     print c
...     printsub(t)
...     c += 1
```

Informatics 2A: Tutorial Sheet 3 - SOLUTIONS

JOHN LONGLEY

1. Suppose $a \in \Sigma$.

- (a) The string aaa can be derived in two ways:
This ambiguity is harmless since both trees define the language $\{aaa\}$.
- (b) The string aa^* can also be derived in two ways.
This ambiguity is harmful since one parse tree defines the regular language $\{a^n \mid n \geq 1\}$ and the other defines $\{a^{2n} \mid n \geq 0\}$. Note that the first parsing is the one that respects the usual precedence conventions.

2. (a) The idea is that we jump to the second state once the b's start coming. Formally, the control states are $Q = \{q_a, q_b\}$, the stack alphabet is $\Gamma = \{\perp\}$, and we include the following transitions:

$$\begin{array}{lcl} q_a & \xrightarrow{\$, \perp : \epsilon} & q_a \\ q_a & \xrightarrow{a, \perp : \perp \perp} & q_a \\ q_a & \xrightarrow{b, \perp : \epsilon} & q_b \\ q_b & \xrightarrow{b, \perp : \epsilon} & q_b \\ q_b & \xrightarrow{\$, \perp : \epsilon} & q_b \end{array}$$

- (b) Let $\Gamma = \{(\,, [\,, \perp\}$. Consider a single state with the following self-transitions.

$$\begin{array}{l} (\,, x : (x \text{ for each } x \in \Gamma \\ [\,, x : [x \text{ for each } x \in \Gamma \\), (: \epsilon \\], [: \epsilon \\ \$, \perp : \epsilon \end{array}$$

3. (a)

Operation	Input remaining	Stack state
	(n * n)\$	Exp
Lookup (,Exp	(n * n)\$	(Exp)
Match (	n * n)\$	Exp)
Lookup n, Exp	n * n)\$	n Ops)
Match n	* n)\$	Ops)
Lookup *, Ops	* n)\$	* n Ops)
Match *	n)\$	n Ops)
Match n	)\$	Ops)
Lookup), Ops	)\$	)
Match)	\$	STACK EMPTY
		AT END OF STRING:
		SUCCESS!

- (b) • For $()$, the parser will encounter a blank table entry at $)$, Exp.
Message: “) Found where expression expected.”

- For n), the stack will empty before end of input is reached.
Message: “) Found after end of expression.”
- For n^* , the end of input will be reached with n Ops still on the stack, and the parser gets stuck since the top of the stack is a terminal n no different from $\$$.
Message: “End of input found where numeric literal expected.”

(c)

$$\begin{aligned} \text{Exp} &\rightarrow \text{ExpA Ops} \\ \text{Ops} &\rightarrow \epsilon \mid * \text{ExpA Ops} \\ \text{ExpA} &\rightarrow n \mid (\text{Exp}) \end{aligned}$$

(Other solutions are possible.)

4. (a) $E = \{\text{OptMinus}, \text{TimesOps}, \text{PlusOps}\}$

- (b) $\text{First}(\text{OptMinus}) = \{-, \epsilon\}$
 $\text{First}(\text{TimesOps}) = \{*, \epsilon\}$
 $\text{First}(\text{PlusOps}) = \{+, \epsilon\}$
 $\text{First}(\text{Exp}) = \text{First}(\text{Cond}) = \text{First}(\text{TimesExp}) = \{-, n\}$

- (c) $\text{Follow}(\text{Cond}) = \{\$\}$
 $\text{Follow}(\text{Exp}) = \{\$, ==\}$
 $\text{Follow}(\text{PlusOps}) = \{\$, ==\}$
 $\text{Follow}(\text{TimesExp}) = \{\$, ==, +\}$
 $\text{Follow}(\text{TimesOps}) = \{\$, ==, +\}$
 $\text{Follow}(\text{OptMinus}) = \{n\}$

- (d) The grammar is indeed LL(1)!

Parse table as follows.

	n	+	*	-	==	\$
Cond	Exp==Exp			Exp==Exp		
Exp	TimesExp PlusOps			TimesExp PlusOps		
TimesExp	OptMinus n TimesOps			OptMinus n TimesOps		
OptMinus	ϵ			-		
TimesOps		ϵ	* n TimesOps		ϵ	ϵ
PlusOps		+ TimesExp PlusOps			ϵ	ϵ

Informatics 2A: Tutorial Sheet 4 - SOLUTIONS

JOHN LONGLEY

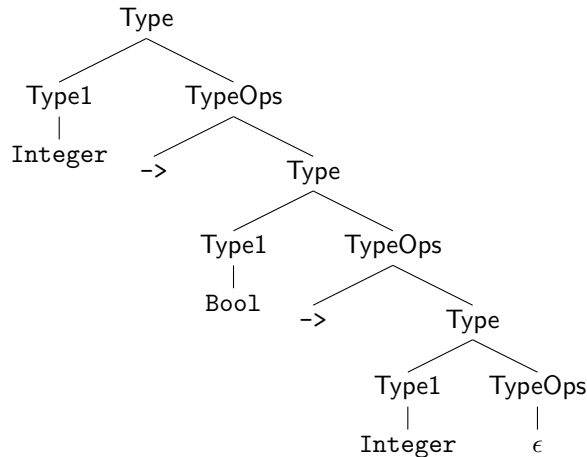
1. A possible LL(1) grammar is as follows (note the distinction between ϵ and $\epsilon!$):

$$\begin{aligned}
 \text{RegExp} &\rightarrow \text{RegExp1 PlusOps} \\
 \text{PlusOps} &\rightarrow \epsilon \mid + \text{RegExp1 PlusOps} \\
 \text{RegExp1} &\rightarrow \text{RegExp2 ConcatOps} \\
 \text{ConcatOps} &\rightarrow \epsilon \mid \text{RegExp2 ConcatOps} \\
 \text{RegExp2} &\rightarrow \text{RegExp3 StarOps} \\
 \text{StarOps} &\rightarrow \epsilon \mid * \text{StarOps} \\
 \text{RegExp3} &\rightarrow \text{Atom} \mid (\text{RegExp}) \\
 \text{Atom} &\rightarrow \text{sym} \mid \emptyset \mid \epsilon
 \end{aligned}$$

For the sake of completeness, the parse table for this is:

	sym/ $\emptyset$ / ϵ	+	*	(	)	\$
RegExp	RegExp1 PlusOps			RegExp1 PlusOps		
PlusOps		+RegExp1 PlusOps				ϵ ϵ
RegExp1	RegExp2 ConcatOps			RegExp2 ConcatOps		
ConcatOps	RegExp2 ConcatOps	ϵ		RegExp2 ConcatOps	ϵ ϵ	
RegExp2	RegExp3 StarOps			RegExp3 StarOps		
StarOps	ϵ	ϵ	* StarOps	ϵ	ϵ ϵ	
RegExp3	Atom			(RegExp)		
Atom	sym/ $\emptyset$ / ϵ					

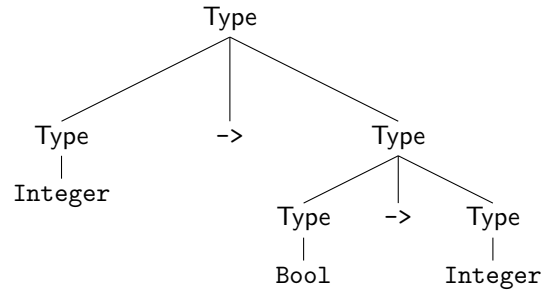
2. (a) The parse tree is:



- (b) By *abstract syntax tree* we mean the intended parse tree of the abstract syntax grammar. In this case it is the parse tree that implicitly brackets the type as

$\text{Integer} \rightarrow (\text{Bool} \rightarrow \text{Integer})$

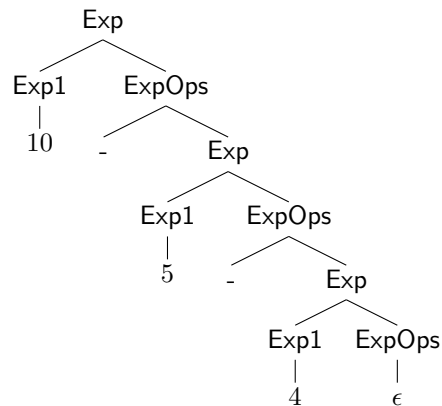
which is the correct bracketing by the Haskell convention that the $\rightarrow$ operation associates to the right. The required tree is:



(c)

$$\begin{aligned} \text{Exp} &\rightarrow \text{Exp1 ExpOps} \\ \text{Exp1} &\rightarrow n \mid (\text{Exp}) \\ \text{ExpOps} &\rightarrow \epsilon \mid + \text{Exp} \mid - \text{Exp} \end{aligned}$$

(d) The parse tree is:

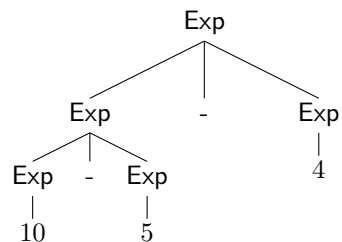


(It's OK to put n in place of the numbers. We can view n as a terminal in the grammar, in which case n should label the parse tree; or, as above, we can view n as standing for the class of all numeric terminals.)

(e) This time, standard conventions require the abstract syntax tree that implicitly brackets the expression as

$$(10 - 5) - 4$$

This leads to the abstract syntax tree below.

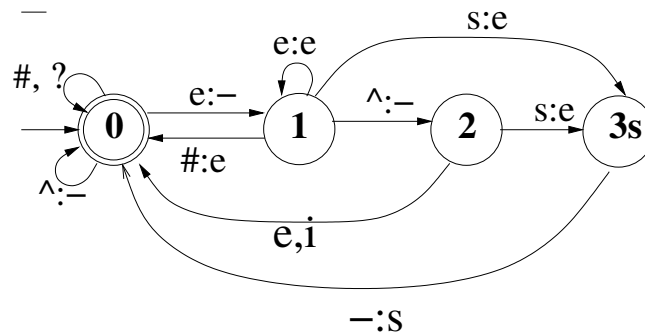


- (f) The difference is that structurally similar concrete parse trees are dealt with in different ways in the translation to abstract syntax tree. The concrete syntax does not distinguish between right-associativity and left-associativity of binary operations. Rather, irrespective of the associativity properties of the operations, expressions are concretely parsed as a *list* of top-level operations. The right-associativity of $\rightarrow$ is catered for by translating concrete parse trees to abstract syntax trees in one way. The left-associativity of $+$ and $-$ is handled by translating in a different way.

The main point to take away from this is: when using LL(1) parse technology, the choice of whether to implement infix operations as left-associative or right-associative is made in the translation from concrete syntax to abstract syntax, it is not made in the formulation of the grammar for the concrete syntax.

3. The following transducer does the job. In the state '3s', we have used s as a sample letter distinct from i and e; the machine should contain a state like this (with similar transitions in and out) for every such letter.

We write '?' to mean 'any letter except e'. For typesetting reasons, we have written '-' in place of ϵ .



The state 3s illustrates a possible approaches to solving the following problem: given any input letter X other than e or i, output e followed by X . (In practice, one would probably adopt a slightly different approach to this problem: e.g. use a version of finite state transducers that allows for a bit of lookahead.)

[Actually, we do also want a state 3i which I haven't shown: this one will have a transition from state 1 but not from state 2.]

Informatics 2A: Tutorial Sheet 5 Solutions

ADAM LOPEZ

1. (a) For the purpose of a rough calculation, we can approximate the frequency graph by the curve $y = c/x$ for a suitable constant c . The total number of tokens will then be

$$\int_1^{10000} c/x \, dx = c[\ln x]_1^{10000} = 4c \ln 10 = 100000$$

(and this fixes the value of c). So to obtain half the total number of tokens, we clearly want to take the 100 most common word types:

$$\int_1^{100} c/x \, dx = c[\ln x]_1^{100} = 2c \ln 10$$

- (b) From the above, we have $c \approx 10857$. So the frequency of *about* is roughly $10857/60 \approx 181$.
2. Here's one way to tag the text, based on the Penn Treebank tagging guidelines:

I/PRP was/VBD walking/VBG down/IN the/DT high/JJ street/NN
yesterday/NN when/CC I/PRP noticed/VBD an/DT old/JJ
man/NN acting/VBG suspiciously/RB . He/PRP was/VBD peer-
ing/VBG into/IN various/JJ shop/NN windows/NN and/CC
writing/VBG things/NNS in/IN a/DT notebook/NN . When/WRB
he/PRP spotted/VBD me/PRP, he/PRP stuffed/VBD the/DT
notebook/NN into/IN his/PRP\$ pocket/NN and/CC wandered/VBD
off/RP ./.

Here's how the Stanford tagger tags it:

I/PRP was/VBD walking/VBG down/RP the/DT high/JJ street/NN
yesterday/NN when/WRB I/PRP noticed/VBD an/DT old/JJ
man/NN acting/VBG suspiciously/RB ./ . He/PRP was/VBD
peering/VBG into/IN various/JJ shop/NN windows/NNS and/CC
writing/VBG things/NNS in/IN a/DT notebook/NN ./ . When/WRB
he/PRP spotted/VBD me/PRP ,/, he/PRP stuffed/VBD the/DT
notebook/NN into/IN his/PRP\$ pocket/NN and/CC wandered/VBD
off/RP ./.

You can see it sometimes makes mistakes, for example, denoting “down” as a particle.

Here is the Penn treebank POS tagset if needed for discussion:

1.	CC	Coordinating conjunction
2.	CD	Cardinal number
3.	DT	Determiner
4.	EX	Existential there
5.	FW	Foreign word
6.	IN	Preposition or subordinating conjunction
7.	JJ	Adjective
8.	JJR	Adjective, comparative
9.	JJS	Adjective, superlative
10.	LS	List item marker
11.	MD	Modal
12.	NN	Noun, singular or mass
13.	NNS	Noun, plural
14.	NNP	Proper noun, singular
15.	NNPS	Proper noun, plural
16.	PDT	Predeterminer
17.	POS	Possessive ending
18.	PRP	Personal pronoun
19.	PRP\$	Possessive pronoun
20.	RB	Adverb
21.	RBR	Adverb, comparative
22.	RBS	Adverb, superlative
23.	RP	Particle
24.	SYM	Symbol
25.	TO	to
26.	UH	Interjection
27.	VB	Verb, base form
28.	VBD	Verb, past tense
29.	VBG	Verb, gerund or present participle
30.	VCN	Verb, past participle
31.	VBP	Verb, non-3rd person singular present
32.	VBZ	Verb, 3rd person singular present
33.	WDT	Wh-determiner
34.	WP	Wh-pronoun
35.	WP\$	Possessive wh-pronoun
36.	WRB	Wh-adverb

For most words here the tagging is straightforward, but the following points might be discussed:

- If *high street* were regarded as a compound noun, the tagging would be high/NN street/NN.
- One might very reasonably want to tag *yesterday* as a temporal adverb (RB). The Penn guidelines, however, say that it should be treated as a noun (even in contexts like the above), pointing out e.g. that it admits a possessive form *yesterday's news*.
- We have tagged the first *when* as a coordinating conjunction, and the second as a Wh-adverb, though it is not entirely clear whether this accords with Penn Treebank policy.

3. We only have to tag the words *old* and *man*, since the tagging of the other words is fixed. Proceeding from left to right, we see that if *old* is preceded by a DT, its most likely POS is Adj, while if *man* is preceded by Adj, its most likely POS is N.

(This is admittedly a rather weak example, in that the tagging of *man* would be the same whatever preceded it!)

4. The Viterbi matrix is as follows:

	the	old	man	the	lifeboats
DT	$.4 \times .5 = .2$	0	0	$.00096 \times .4 \times .5 = .000192$	0
N	0	$.2 \times .6 \times .2 = .024$	$.032 \times .5 \times .3 = .0048$	0	etc.
V	0	0	$.024 \times .4 \times .1 = .00096$	0	0
Adj	0	$.2 \times .4 \times .4 = .032$	0	0	0

Thus the most probable tagging is:

The/DT old/N man/V the/DT lifeboats/N

(The backtrace pointers can be read off from the above matrix in an ad hoc fashion: e.g. in the cell for (man,N), the first factor is .032 which comes from the cell for (the,Adj).)

Informatics 2A: Tutorial Sheet 6 Solutions

ADAM LOPEZ

1. (a) One possible equivalent CNF grammar is as follows:

$$\begin{aligned} S &\rightarrow NP VP & NP &\rightarrow Det N \\ S &\rightarrow Pro VPP & PP &\rightarrow Pre NP \\ VPP &\rightarrow VP PP & VP &\rightarrow ate \\ VP &\rightarrow V NP & V &\rightarrow ate \\ Pro &\rightarrow I \end{aligned}$$

plus the original lexical rules for Det, N, Pre.

- (b) Using the above CNF grammar, the CYK chart in matrix format would be:

	1	2	3	4	5	6	7
0	Pro						S
1		V,VP		VP			VPP
2			Det	NP			
3				N			
4					Pre		PP
5						Det	NP
6							N

- (c) There is just one complete parse; its tree is represented by

$$(S (Pro I) (VPP (VP (V ate)(NP(Det the)(N salad))) (PP (Pre with)(NP(Det a)(N fork)))))$$

- (d) If we record the steps used to transform our grammar into CNF, it's clear how each step yields a mapping from 'new' parse trees to 'old' ones. For instance, if we replace the rule $S \rightarrow Pro VP PP$ by the two rules $S \rightarrow Pro VPP$, $VPP \rightarrow VP PP$, it's clear that a tree of the form

$$(S (Pro subtree1) (VPP (VP subtree2)(PP subtree3)))$$

maps back to

$$(S (Pro subtree1) (VP subtree2)(PP subtree3))$$

- (e) We could add a new rule

$$NP \rightarrow NP PP$$

This would add an NP entry to cell (2,7), hence a VP entry to (1,7). However, no new S entry would be added to (0,7), so there is still just one complete parse.

2. (a) The Earley parse table is as follows. (P=Predictor, S=Scanner, C=Completer.)

$S \rightarrow \bullet NP VP$	$[0,0]$	P
$S \rightarrow \bullet Pro V$	$[0,0]$	P
$NP \rightarrow \bullet Pro$	$[0,0]$	P
$Pro \rightarrow I \bullet$	$[0,1]$	S
$NP \rightarrow Pro \bullet$	$[0,1]$	C
$S \rightarrow NP \bullet VP$	$[0,1]$	C
$S \rightarrow Pro \bullet V$	$[0,1]$	C
$VP \rightarrow \bullet V Det N$	$[1,1]$	P
$V \rightarrow saw \bullet$	$[1,2]$	S
$S \rightarrow Pro V \bullet$	$[0,2]$	C
$VP \rightarrow V \bullet Det N$	$[1,2]$	C
$Det \rightarrow the \bullet$	$[2,3]$	S
$VP \rightarrow V Det \bullet N$	$[1,3]$	C
$N \rightarrow saw \bullet$	$[3,4]$	S
$VP \rightarrow V Det N \bullet$	$[1,4]$	C
$S \rightarrow NP VP \bullet$	$[0,4]$	C

- (b) The Earley algorithm never tries assigning the ‘wrong’ part of speech to either occurrence of *saw*, since in both cases this is precluded by what comes before it. The CYK algorithm will try the other possibilities, but only the correct assignments will be found to contribute to an overall parse.

Informatics 2A: Tutorial Sheet 7 Solutions

ADAM LOPEZ

- The probabilistic CYK-style chart is as follows (note this is not ‘pure CYK’ as the grammar is not in Chomsky Normal Form). The derivations of cell probabilities are shown in selected cases.

	I	make	her	duck
I	Subj 0.3		S $1.0 \times 0.3 \times 0.06 = 0.018$	S $1.0 \times 0.3 \times 0.06 = 0.018$
make		V 0.6	VP $0.5 \times 0.6 \times 0.2 = 0.06$	VP $0.5 \times 0.6 \times 0.2 = 0.06$
her			Det 1.0, Obj 0.2	NP 0.25, Obj 0.2, Subj 0.175, Small $1.0 \times 0.2 \times 0.4 = 0.08$
duck				V 0.4, N 0.5, NP 0.25, Obj 0.2, Subj 0.175

The main interest comes in the cell (make,duck). There are three competing analyses of ‘make her duck’ as a VP, with probabilities as follows:

V Obj	$0.5 \times 0.6 \times 0.2$	$= 0.06$
V Obj Obj	$0.3 \times 0.6 \times 0.2 \times 0.2$	$= 0.0072$
V Small	$0.2 \times 0.6 \times 0.08$	$= 0.0096$

So the first of these is the most probable, and is the only one we need record in the chart. Note that for the purpose of computing the most probable analysis of the whole sentence, this is the only analysis of ‘make her duck’ that need be considered.

Thus, the most probable overall parse is

(S (Subj *I*) (VP (V *make*) (Obj (NP (Det *her*) (N *duck*))))))

with probability 0.018.

- (a) Here are the rules and their probabilities (which we write as rational fractions):

S $\rightarrow$ NP VP	(4/4)	V $\rightarrow$ saw	(3/4)
VP $\rightarrow$ VP PP	(1/5)	V $\rightarrow$ is	(1/4)
VP $\rightarrow$ V NP	(4/5)	Det $\rightarrow$ the	(7/9)
NP $\rightarrow$ Det N	(9/14)	Det $\rightarrow$ a	(2/9)
NP $\rightarrow$ NP PP	(2/14)	N $\rightarrow$ man	(3/9)
NP $\rightarrow$ he	(1/14)	N $\rightarrow$ girl	(2/9)
NP $\rightarrow$ she	(1/14)	N $\rightarrow$ distance	(1/9)
NP $\rightarrow$ here	(1/14)	N $\rightarrow$ guitar	(1/9)
PP $\rightarrow$ P NP	(3/3)	N $\rightarrow$ telescope	(1/9)
P $\rightarrow$ from	(1/3)	N $\rightarrow$ flowers	(1/9)
P $\rightarrow$ with	(2/3)		

- (b) Although not asked for by the question, here is the parse chart (as a matrix):

	He	saw	the	man	with	the	telescope
He	NP(1/14)			S(1/140)			S(1/18900)
saw		V(3/4)		VP(1/10)			VP ₁ (1/1350), VP ₂ (1/1890)
the			Det(7/9)	NP(1/6)			NP(1/1134)
man				N(1/3)			
with					P(2/3)	PP(1/27)	
the						Det(7/9)	NP(1/18)
tel.							N(1/9)

Here $VP_1 \rightarrow VP PP$ and $VP_2 \rightarrow V NP$. Since VP_1 has the higher probability, strictly speaking the chart should only include this one, and this is compute the probability of S in the top-right cell.

Thus, the most likely parse is

(S (NP he) (VP (VP (V saw) (NP (Det the) (N man)))) (PP (P with) (NP (Det the) (N telescope)))))

with probability 1/18900. The other parse has probability 1/26460.

(c) There are two competing analyses for the sentence so far:

(S (NP (Det the)(N girl)) (VP (V saw) (NP ...)))
(S (NP (Det the)(N girl)) (VP (VP (V saw) (NP ...)) (PP ...)))

In either case, ‘saw’ must be followed by an NP, and the most probable NP is ‘the man’, with probability 1/6. Plugging this into both the above trees, we see that the second already contains all the rule applications in the first tree and more besides, so even without considering the expansion of PP, we see that the first structure is more probable:

(S (NP (Det the)(N girl)) (VP (V saw) (NP (Det the)(N man)))))

Thus the most likely completion is

The girl saw the man

Informatics 2A: Tutorial Sheet 8 Solutions

ADAM LOPEZ

1. The question is somewhat open-ended, but one solution is as follows. We consider the following three attributes:

Person:	values 1,2,3, ranged over by x
Number:	values s,p, ranged over by y
Gender:	values m,f,n, ranged over by z

The following parameterized rules suffice:

S	→	SPro[x,y,z] Vstem Vsuf[x,y] ReflPro[x,y,z]
Vstem	→	<i>love prepare congratulate</i>
SPro[1,s,z]	→	<i>I</i>
SPro[1,p,z]	→	<i>We</i>
SPro[2,y,z]	→	<i>You</i>
SPro[3,s,m]	→	<i>He</i>
SPro[3,s,f]	→	<i>She</i>
SPro[3,s,n]	→	<i>It</i>
SPro[3,p,z]	→	<i>They</i>
Vsuf[3,s]	→	<i>-s</i>
Vsuf[1,s]	→	ϵ , etc.
ReflPro[1,s,z]	→	<i>myself</i>
ReflPro[2,s,z]	→	<i>yourself</i>
ReflPro[3,s,m]	→	<i>himself</i>
ReflPro[3,s,f]	→	<i>herself</i>
ReflPro[3,s,n]	→	<i>itself</i>
ReflPro[1,p,z]	→	<i>ourselves</i>
ReflPro[2,p,z]	→	<i>yourselves</i>
ReflPro[3,p,z]	→	<i>themselves</i>

2. We want a single constant, Jumbo, and predicates with arities as follows:

elephant/1 mammal/1 owns/2 song/1 sings_to/3 danced/1

We also have a binary equality predicate as a standard part of our FOPL machinery.

The given sentences may be translated into FOPL as follows:

- *Jumbo is an elephant:* elephant(Jumbo)
- *An elephant is a mammal:* this could arguably have either of the following meanings.

$$\forall x. \text{elephant}(x) \Rightarrow \text{mammal}(x)$$

$$\exists x. \text{elephant}(x) \wedge \text{mammal}(x)$$

- *Every elephant has an owner:*

$$\forall x. \text{elephant}(x) \Rightarrow \exists y. \text{owns}(y, x)$$

or possibly

$$\exists y. \forall x. \text{elephant}(x) \Rightarrow \text{owns}(y, x)$$

- *Everyone who owns an elephant sings it a song:*

$$\forall x. \forall y. (\text{owns}(x, y) \wedge \text{elephant}(y)) \Rightarrow \exists z. (\text{song}(z) \wedge \text{sings_to}(x, z, y))$$

or possibly

$$\forall x. \forall y. \exists z. (\text{owns}(x, y) \wedge \text{elephant}(y)) \Rightarrow (\text{song}(z) \wedge \text{sings_to}(x, z, y))$$

- *Only one elephant danced.*

$$\exists x. \text{elephant}(x) \wedge \text{danced}(x) \wedge (\forall y. (\text{elephant}(y) \wedge \text{danced}(y)) \Rightarrow y = x)$$

- *Every elephant did not dance:* there is a scoping ambiguity here (cf. the expression *All is not lost.*) Either

$$\neg(\forall x. \text{elephant}(x) \Rightarrow \text{danced}(x))$$

or

$$\forall x. \text{elephant}(x) \Rightarrow \neg \text{danced}(x)$$

3. The semantics of the given phrases are as follows. For the first two examples, we show the β -reductions steps; for the others, we show only the result after β -reduction.

- John runs:

$$(\lambda P. P(\text{John}))(\lambda x. \text{run}(x)) \rightarrow_{\beta} (\lambda x. \text{run}(x))(\text{John}) \rightarrow_{\beta} \text{run}(\text{John})$$

- likes ice-cream:

$$\begin{aligned} & \lambda x. (\lambda P. P(\text{Ice-cream}))((\lambda x. \lambda y. \text{like}(x, y))(x)) \\ & \rightarrow_{\beta} \lambda x. (\lambda P. P(\text{Ice-cream}))(\lambda y. \text{like}(x, y)) \\ & \rightarrow_{\beta} \lambda x. (\lambda y. \text{like}(x, y))(\text{Ice-cream}) \rightarrow_{\beta} \lambda x. \text{like}(x, \text{Ice-cream}) \end{aligned}$$

- John likes ice-cream: $\text{like}(\text{John}, \text{Ice-cream})$
- an ice-cream: $\lambda P. \exists x. \text{ice-cream}(x) \wedge P(x)$
- likes an ice-cream: $\lambda y. \exists x. \text{ice-cream}(x) \wedge \text{like}(y, x)$
- John likes an ice-cream: $\exists x. \text{ice-cream}(x) \wedge \text{like}(\text{John}, x)$
- every cat: $\lambda P. \forall x. \text{cat}(x) \Rightarrow P(x)$
- every cat likes ice-cream: $\forall x. \text{cat}(x) \Rightarrow \text{like}(\text{cat}, \text{Ice-cream})$
- every cat likes an ice-cream: $\forall x. \text{cat}(x) \Rightarrow \exists y. \text{ice-cream}(y) \wedge \text{like}(x, y)$

Informatics 2A: Tutorial Sheet 9 - SOLUTIONS

ALEX SIMPSON AND JOHN LONGLEY

1. (a) The language processing pipeline is the sequence of stages in which a document or phrase of a language is converted from raw input (e.g. speech; program text) to final result (e.g. understanding; program execution.) For the typical structure of such pipelines in the case of formal and natural languages, see the slides from Lecture 2. (These would not need to be reproduced exactly to get full marks.)
- (b) Perhaps the most important difference between processing NLS and FLs is that for NLS, *ambiguity* can enter in at many stages of the pipeline, whereas in FLs, this is usually avoidable by design. This means that whereas for FLs the pipeline typically works in a very pure way — the unique output from each stage provides all that is needed for later stages — in NLS things are more complex: either each stage of the pipeline delivers multiple possible analyses to the next stage (perhaps ranked by probability), or some kind of feedback or interdependence between stages is built in.
2. (a) Equations:

$$\begin{aligned} X_1 &= aX_2 + bX_1 + \epsilon \\ X_2 &= aX_1 + bX_3 \\ X_3 &= aX_2 + bX_2 \end{aligned}$$

- (b) Plugging equation for X_3 into that for X_2 we get

$$\begin{aligned} X_2 &= aX_1 + b(aX_2 + bX_2) \\ &= aX_1 + b(a + b)X_2 \\ &= (b(a + b))^* aX_1 \quad \text{by Arden's Rule} \end{aligned}$$

Now substituting into equation X_1 , we get

$$\begin{aligned} X_1 &= a(b(a + b))^* aX_1 + bX_1 + \epsilon \\ &= (a(b(a + b))^* a + b)X_1 + \epsilon \\ &= (a(b(a + b))^* a + b)^* \quad \text{by Arden's Rule} \end{aligned}$$

Thus the regular expression for the language is

$$(a(b(a + b))^* a + b)^*$$

3. (a)

transition	control state	unread input	stack
	$q1$	$abba$	$\perp$
$q1 \xrightarrow{a, \perp : a \perp} q1$	$q1$	bba	$a \perp$
$q1 \xrightarrow{b, a : ba} q1$	$q1$	ba	$ba \perp$
$q1 \xrightarrow{\epsilon, b : b} q2$	$q2$	ba	$ba \perp$
$q2 \xrightarrow{b, b : \epsilon} q2$	$q2$	a	$a \perp$
$q2 \xrightarrow{a, a : \epsilon} q2$	$q2$	ϵ	$\perp$
$q2 \xrightarrow{\epsilon, \perp : \epsilon} q2$	$q2$	ϵ	ϵ

String accepted because empty stack reached at end of input.

(b) The language is:

$$\{ww' \mid w \in \{a, b\}^* \text{ and } w' \text{ is the reverse of } w\}$$

In other words, it is the language of even-length palindromes.

4. (a) The probabilistic CYK chart is as follows:

	<i>fat</i>	<i>ducks</i>	<i>fish</i>
<i>fat</i>	N (.2), A (1.0), NP (.1)	S (.02), NP (.1)	S (.03)
<i>ducks</i>		N (.4), V (.4), NP (.2), VP (.2)	S (.06), VP (.04)
<i>fish</i>			N (.4), V (.6), NP (.2), VP (.3)

For the top right cell, there is also a competing analysis of S as (NP fat) (VP ducks fish), but this has the lower probability .004.

(b) From the construction of the above chart, the most probable parse (with probability .03) is seen to be

$$(S (NP (A \textit{fat}) (NP (N \textit{ducks})))) (VP (V \textit{fish}))$$

5. (a) The semantics of *outside Scotland* (a phrase of category PrP) is obtained as:

$$\begin{aligned} & \textit{outside.Sem}(\textit{Scotland.Sem}) \\ = & (\lambda y. \lambda z. \neg \text{In}(z, y))(\text{Scotland}) \\ \rightarrow_{\beta} & (\lambda z. \neg \text{In}(z, \text{Scotland})) \end{aligned}$$

The semantics of *city outside Scotland* (of category NP) is therefore

$$\begin{aligned} & \lambda x. \textit{city.Sem}(x) \wedge (\textit{outside Scotland}).\textit{Sem}(x) \\ = & \lambda x. ((\lambda z. \textit{city}(z))(x) \wedge (\lambda z. \neg \text{In}(z, \text{Scotland}))(x)) \\ \rightarrow_{\beta} & \lambda x. \textit{city}(x) \wedge \neg \text{In}(x, \text{Scotland}) \end{aligned}$$

Thus the semantic of the complete sentence is

$$\begin{aligned} & (\textit{city outside Scotland}).\textit{Sem}(\textit{London.Sem}) \\ = & (\lambda x. \textit{city}(x) \wedge \neg \text{In}(x, \text{Scotland}))(\text{London}) \\ \rightarrow_{\beta} & \textit{city}(\text{London}) \wedge \neg \text{In}(\text{London}, \text{Scotland}) \end{aligned}$$

(b) The second sentence yields the logical formula

$$\neg(\textit{city}(\text{London}) \wedge \text{In}(\text{London}, \text{Scotland}))$$

(c) The first formula implies the second by the laws of propositional logic. But not conversely: e.g. if London were not a city at all, the second formula would hold but not the first.

6. (a) Underlining the sequence to be expanded on the next line:

$$\begin{aligned}
 \underline{S} &\Rightarrow A_{\text{hooray}} \underline{S} E \\
 &\Rightarrow A_{\text{hooray}} \underline{A_{\text{hip}} E} E \\
 &\Rightarrow \underline{A_{\text{hooray}} \text{hip}} ! E \\
 &\Rightarrow \underline{\text{hip } A_{\text{hooray}} !} E \\
 &\Rightarrow \text{hip hip ! } \underline{A_{\text{hooray}} E} \\
 &\Rightarrow \text{hip hip ! hooray !}
 \end{aligned}$$

- (b) The language L consists of all phrases of the form:

$$w_1^n ! w_2^{n-1} ! w_3^{n-2} ! \dots w_{n-1}^2 ! w_n !$$

where $n \geq 1$, $w_1, \dots, w_n$ are English words and w^k means that the word w is repeated k times in a row, e.g., hip^2 represents hip hip.

- (c) The language is context sensitive. This follows because the grammar is an example of a noncontracting grammar.



- ▶ [University Homepage](#)
- ▶ [School Homepage](#)
- ▶ [School Contacts](#)
- ▶ [School Search](#)

Informatics 2A: Processing Formal and Natural Languages (2016-17)

Inf2A Webpages

- [Lecture Details and Course Schedule \(including lecture slides\)](#)
- [Official Course Descriptor](#)
- [Inf2A on PATH](#)
- [Year Guide](#) (common to all Informatics 2 courses)
- [Staff and Course Representatives](#)
- [Tutorial Groups: Times, Places and Allocations](#)
- [Lab Session Groups](#)
- [Tutorial and Lab Materials](#)
- [Assignments](#)
- [Reading Material](#)
- [Course Discussion Forum](#)
- [Past exam papers and solutions](#)

Tutorials and Lab Sessions

Tutorial group allocations will be confirmed within the first two weeks of the course. If you need to change your tutorial group, please email the Course Secretary Kendal Reid using the ITO email (ito@inf.ed.ac.uk) to arrange this.

Lab sessions will run in Weeks 3,4,5,6,10 and 11. In order to keep the numbers even, please try to attend the session you are allocated to if possible; otherwise, you may just turn up to another session without prior arrangement.

Software

As part of this course, you will learn how to program in [Python](#), and how to use [NLTK](#), the Natural Language Toolkit. The [Lab Materials](#) page has worksheets to help you get to grips with these. Another relevant piece of software is [JFLAP](#), the interactive formal languages and automata package.

Exam

The Inf2A exam will take place in December (resit in August). The exam date, time and venue will be publicized here when available (these are set by Registry, not by the School of Informatics).

It will be a closed book exam, similar in format to last year's Inf2A exam. Calculators are permitted, but you must provide your own, and it must be one of an [approved list of models](#) specified by College.

Feedback

The course will provide students with both *summative* feedback on their performance (i.e. feedback on assessed coursework that contributes to the final course mark) and *formative* feedback (i.e. feedback on aspects of the course not contributing to the final mark). Summative feedback will include:

1. Printed feedback on each assignment, including automatic tests run on the student's submission, the results of such tests, and additional comments by a human marker where appropriate. This will be made available to students within two weeks of the assignment submission deadline.
2. Oral feedback on Assignment 1 submissions, given at the lab sessions in Weeks 9 and 10.

Formative feedback will include:

1. Oral feedback given by tutors in tutorials on students' solutions to tutorial problems (this will help students to prepare for similar questions in the course exam).
2. Oral feedback given by demonstrators in lab sessions on students' solutions to the exercises on the Python/NLTK worksheets (this will help students to develop the skills required for Assignment 2).

Class Reps

If you wish to make a comment to the lecturers anonymously, you may do so through the class reps for Year 2 courses. Contact details are given on the [Student Representation](#) page.

[Home](#) : [Teaching](#) : [Courses](#)

Informatics Forum, 10 Crichton Street, Edinburgh, EH8 9AB, Scotland, UK
Tel: +44 131 651 5661, Fax: +44 131 651 1426, E-mail: school-office@inf.ed.ac.uk
Please [contact our webadmin](#) with any comments or corrections. [Logging and Cookies](#)
Unless explicitly stated otherwise, all material is copyright © The University of Edinburgh



Informatics 2A: Course Schedule and Lecture Slides, Autumn 2016

Lectures will take place at the following times and places:

- Tuesdays: 16:10 - 17:00, David Hume Tower, Lecture Hall C
- Thursdays: 17:10 - 18:00, Appleton Tower, Lecture Theatre 5
- Fridays: 16:10 - 17:00, David Hume Tower, Lecture Hall C

Links to the lecture slides will become live at the time of the lecture in question. Should you wish to look at the material in advance of the lecture, feel free to look at [last year's slides](#). These are not guaranteed to be identical to this year's slides, but in practice the content will be mostly the same.

Week	Lectures	Tutorials	Practicals	Assignments	Readings
1	Sep 20: Lecture 1: Introduction and Course Administration (JL+AL) Sep 22: Lecture 2: Course Roadmap (JL+AL). VIDEO Sep 23: Lecture 3: Finite Automata (JL). VIDEO	--	--	--	Kozen chapters 2,3,5,6; J&M[2nd Ed] chapters 1, 2 (sec 2.2), 16 (intro and sec 16.1); Wikipedia: Chomsky (see Note 1)
2	Sep 27: Lecture 4: Constructions on Finite Automata (JL). VIDEO Sep 29: Lecture 5: Regular Expressions and Kleene's Theorem (JL). VIDEO Sep 30: Lecture 6: Applications to String and Pattern Matching (JL). VIDEO	--	--	--	Kozen chapter 3, 4, 11, 12, 2

3	Oct 04: Lecture 7: Lexing and Other Applications (JL). VIDEO Oct 06: Lecture 8: The Pumping Lemma: limitations of regular languages (JL). VIDEO Oct 07: Lecture 9: Context-free Languages (JL). VIDEO	Tutorial 1	Lab 1: Introduction to Python/NLTK, Part I	--	Kozen chapter 5, 9; J&M chapter 2; chapter 16.2; Vidal et al. 2005
4	Oct 11: Lecture 10: Pushdown Automata (JL). VIDEO Oct 13: Lecture 11: LL(1) Predictive Parsing (JL). VIDEO (sound mostly OK despite mic failure) Oct 14: Lecture 12: Automatic Generation of LL(1) Parsers (JL). VIDEO	Tutorial 2	Lab 2: Introduction to Python/NLTK, Part II	--	Earlier year's lecture notes relevant lectures 10 and 11: Note 9, Note 10, Note 12. Earlier year's tutorial sheet.
5	Oct 18: Lecture 13: Types and Static Type Checking (JL). VIDEO Oct 20: Lecture 14: Fixing Problems with Grammars (JL). VIDEO Oct 21: Lecture 15: Natural language processing, morphology, and finite-state transducers (AL)	Tutorial 3		Oct 18: Assignment 1 issued	Kozen chapters 26 and 27; J&M sections 3.1--3.7

6	Oct 25: Lecture 16: Parts of speech in natural language (AL). VIDEO Oct 27: Lecture 17: Part-of-speech Tagging (AL). VIDEO Oct 28: Lecture 18: Fun with weighed FSTs (AL) VIDEO	Tutorial 4	Lab 3: Support for Assignment 1	--	J&M sections 5.1--5.5, 6.1--6.4; NLTK: Chapters 3 and 5. Penn Treebank tagging guide
7	Nov 01: Lecture 19: Phrase Structure and Parsing as Search (AL) VIDEO Nov 03: Lecture 20: The CYK Algorithm (AL) VIDEO Nov 05: Lecture 21: The Earley Algorithm (AL)	Tutorial 5	--	Nov 01: Assignment 1 deadline (4pm)	J&M Chapters 13.4, 14 (up to 14.5), 16.3--16.4. NLTK: Chapter 8. Rosenberg, <i>The Hardest Natural Languages</i>
8	Nov 08: Lecture 22: Probabilistic Context-free Grammars (AL) VIDEO Nov 10: Lecture 23: Probability; Agreement and Types in Natural Language (AL) VIDEO Nov 11: Lecture 24: Semantics for Natural Languages (AL) VIDEO	Tutorial 6	--	Nov 11: Assignment 2 issued	J&M Chapters 12.9, 14.10, 17--18; NLTK: Chapter 10
9	Nov 15: Lecture 25: Computing Natural Language Semantics (AL) VIDEO Nov 17: Lecture 26: Complexity of Human Languages (AL) VIDEO Nov 18: Lecture 27:	Tutorial 7	Lab 4: Support for Assignment 2, feedback on Assignment 1	--	Lecture Notes: Semantics;

	Complexity (continued); Models of Human Parsing (CCG example) (AL) VIDEO				
10	Nov 22: Lecture 28: Semantics of Programming Languages (JL). VIDEO Nov 24: Lecture 29: Context-sensitive languages (JL). VIDEO Nov 25: Lecture 30: Turing Machines and Linear Bounded Automata (JL). VIDEO	Tutorial 8	Lab 5: Support for Assignment 2	--	Kozen chapter 28, 29, 31, 32; Wikipedia: CSG, Turing
11	Nov 29: Lecture 31: Undecidability (JL) Dec 01: Lecture 32: Revision Lecture (JL+AL) VIDEO	Tutorial 9	Extra labs for Assignment 2 support	Dec 02: Assignment 2 deadline (4pm)	Hauser et al. (2002)

Note 1. To allow for your getting the required texts during Week 1, Chapter 1 from Jurafsky and Martin (2nd Edition) is available [here](#).

[Home](#) : [Teaching](#) : [Courses](#) : [Inf2a](#)

Informatics Forum, 10 Crichton Street, Edinburgh, EH8 9AB, Scotland, UK
Tel: +44 131 651 5661, Fax: +44 131 651 1426, E-mail: school-office@inf.ed.ac.uk
Please [contact our webadmin](#) with any comments or corrections. [Logging and Cookies](#)
Unless explicitly stated otherwise, all material is copyright © The University of Edinburgh

Speech and Language Processing

An Introduction to Natural Language Processing,
Computational Linguistics, and Speech Recognition

Second Edition

Daniel Jurafsky
Stanford University

James H. Martin
University of Colorado at Boulder



Upper Saddle River, New Jersey 07458

Chapter 1

Introduction

Dave Bowman: Open the pod bay doors, HAL.
HAL: I'm sorry Dave, I'm afraid I can't do that.
Stanley Kubrick and Arthur C. Clarke,
screenplay of *2001: A Space Odyssey*

The idea of giving computers the ability to process human language is as old as the idea of computers themselves. This book is about the implementation and implications of that exciting idea. We introduce a vibrant interdisciplinary field with many names corresponding to its many facets, names like **speech and language processing**, **human language technology**, **natural language processing**, **computational linguistics**, and **speech recognition and synthesis**. The goal of this new field is to get computers to perform useful tasks involving human language, tasks like enabling human-machine communication, improving human-human communication, or simply doing useful processing of text or speech.

Conversational agent

One example of a useful such task is a **conversational agent**. The HAL 9000 computer in Stanley Kubrick's film *2001: A Space Odyssey* is one of the most recognizable characters in 20th century cinema. HAL is an artificial agent capable of such advanced language behavior as speaking and understanding English, and at a crucial moment in the plot, even reading lips. It is now clear that HAL's creator, Arthur C. Clarke, was a little optimistic in predicting when an artificial agent such as HAL would be available. But just how far off was he? What would it take to create at least the language-related parts of HAL? We call programs like HAL that converse with humans in natural language **conversational agents** or **dialogue systems**. In this text we study the various components that make up modern conversational agents, including language input (**automatic speech recognition** and **natural language understanding**) and language output (dialogue and response planning and **speech synthesis**).

Dialogue system

Let's turn to another useful language-related task, that of making available to non-English-speaking readers the vast amount of scientific information on the Web in English. Or translating for English speakers the hundreds of millions of Web pages written in other languages like Chinese. The goal of **machine translation** is to automatically translate a document from one language to another. We introduce the algorithms and mathematical tools needed to understand how modern machine translation works. Machine translation is far from a solved problem; we cover the algorithms currently used in the field, as well as important component tasks.

Machine translation

Many other language processing tasks are also related to the Web. Another such task is **Web-based question answering**. This is a generalization of simple Web search, where instead of just typing keywords, a user might ask complete questions, ranging from easy to hard, like the following:

Question answering

- What does "divergent" mean?
- What year was Abraham Lincoln born?
- How many states were in the United States that year?

- How much Chinese silk was exported to England by the end of the 18th century?
- What do scientists think about the ethics of human cloning?

Some of these, such as **definition** questions, or simple **factoid** questions like dates and locations, can already be answered by search engines. But answering more complicated questions might require extracting information that is embedded in other text on a Web page, doing **inference** (drawing conclusions based on known facts), or synthesizing and summarizing information from multiple sources or Web pages. In this text we study the various components that make up modern understanding systems of this kind, including **information extraction**, **word sense disambiguation**, and so on.

Although the subfields and problems we've described above are all very far from completely solved, these are all very active research areas and many technologies are already available commercially. In the rest of this chapter, we briefly summarize the kinds of knowledge that are necessary for these tasks (and others like **spelling correction**, **grammar checking**, and so on), as well as the mathematical models that are introduced throughout the book.

1.1 Knowledge in Speech and Language Processing

What distinguishes language processing applications from other data processing systems is their use of *knowledge of language*. Consider the Unix `wc` program, which counts the total number of bytes, words, and lines in a text file. When used to count bytes and lines, `wc` is an ordinary data processing application. However, when it is used to count the words in a file, it requires *knowledge about what it means to be a word* and thus becomes a language processing system.

Of course, `wc` is an extremely simple system with an extremely limited and impoverished knowledge of language. Sophisticated conversational agents like HAL, machine translation systems, or robust question-answering systems require much broader and deeper knowledge of language. To get a feeling for the scope and kind of required knowledge, consider some of what HAL would need to know to engage in the dialogue that begins this chapter, or for a question-answering system to answer one of the questions above.

HAL must be able to recognize words from an audio signal and to generate an audio signal from a sequence of words. These tasks of **speech recognition** and **speech synthesis** require knowledge about **phonetics and phonology**: how words are pronounced in terms of sequences of sounds and how each of these sounds is realized acoustically.

Note also that unlike Commander Data in “Star Trek”, HAL is capable of producing contractions like *I'm* and *can't*. Producing and recognizing these and other variations of individual words (e.g., recognizing that *doors* is plural) requires knowledge about **morphology**, the way words break down into component parts that carry meanings like *singular* versus *plural*.

Moving beyond individual words, HAL must use structural knowledge to properly string together the words that constitute its response. For example, HAL must know that the following sequence of words will not make sense to Dave, despite the fact that

it contains precisely the same set of words as the original.

(1.1) I'm I do, sorry that afraid Dave I'm can't.

The knowledge needed to order and group words comes under the heading of **syntax**.

Now consider a question-answering system dealing with the following question:

(1.2) How much Chinese silk was exported to Western Europe by the end of the 18th century?

To answer this question, we need to know something about **lexical semantics**, the meaning of all the words (*export* or *silk*) as well as **compositional semantics** (what exactly constitutes *Western Europe* as opposed to Eastern or Southern Europe, what does *end* mean when combined with *the 18th century*). We also need to know something about the relationship of the words to the syntactic structure. For example, we need to know that *by the end of the 18th century* is a temporal end-point and not a description of the agent, as the *by*-phrase is in the following sentence:

(1.3) How much Chinese silk was exported to Western Europe by southern merchants?

We also need the kind of knowledge that lets HAL determine that Dave's utterance is a request for action, as opposed to a simple statement about the world or a question about the door, as in the following variations of his original statement.

REQUEST:	<i>HAL, open the pod bay door.</i>
STATEMENT:	<i>HAL, the pod bay door is open.</i>
INFORMATION QUESTION:	<i>HAL, is the pod bay door open?</i>

Next, despite its bad behavior, HAL knows enough to be polite to Dave. It could, for example, have simply replied *No* or *No, I won't open the door*. Instead, it first embellishes its response with the phrases *I'm sorry* and *I'm afraid*, and then only indirectly signals its refusal by saying *I can't*, rather than the more direct (and truthful) *I won't*.¹ This knowledge about the kind of actions that speakers intend by their use of sentences is **pragmatic** or **dialogue** knowledge.

Another kind of pragmatic or **discourse** knowledge is required to answer the question

(1.4) How many states were in the United States *that year*?

What year is *that year*? To interpret words like *that year*, a question-answering system needs to examine the earlier questions that were asked; in this case, the previous question talked about the year that Lincoln was born. Thus, this task of **coreference resolution** makes use of knowledge about how words like *that* or pronouns like *it* or *she* refer to previous parts of the **discourse**.

To summarize, engaging in complex language behavior requires various kinds of knowledge of language:

- Phonetics and Phonology — knowledge about linguistic sounds

¹ For those unfamiliar with HAL, it is neither sorry nor afraid, nor is it incapable of opening the door. It has simply decided in a fit of paranoia to kill its crew.

- Morphology — knowledge of the meaningful components of words
- Syntax — knowledge of the structural relationships between words
- Semantics — knowledge of meaning
- Pragmatics — knowledge of the relationship of meaning to the goals and intentions of the speaker
- Discourse — knowledge about linguistic units larger than a single utterance

1.2 Ambiguity

Ambiguity
Ambiguous

A perhaps surprising fact about these categories of linguistic knowledge is that most tasks in speech and language processing can be viewed as resolving **ambiguity** at one of these levels. We say some input is **ambiguous** if multiple, alternative linguistic structures can be built for it. Consider the spoken sentence *I made her duck*. Here are five different meanings this sentence could have (see if you can think of some more), each of which exemplifies an ambiguity at some level:

- (1.5) I cooked waterfowl for her.
- (1.6) I cooked waterfowl belonging to her.
- (1.7) I created the (plaster?) duck she owns.
- (1.8) I caused her to quickly lower her head or body.
- (1.9) I waved my magic wand and turned her into undifferentiated waterfowl.

These different meanings are caused by a number of ambiguities. First, the words *duck* and *her* are morphologically or syntactically ambiguous in their part-of-speech. *Duck* can be a verb or a noun, while *her* can be a dative pronoun or a possessive pronoun. Second, the word *make* is semantically ambiguous; it can mean *create* or *cook*. Finally, the verb *make* is syntactically ambiguous in a different way. *Make* can be transitive, that is, taking a single direct object (1.6), or it can be ditransitive, that is, taking two objects (1.9), meaning that the first object (*her*) was made into the second object (*duck*). Finally, *make* can take a direct object and a verb (1.8), meaning that the object (*her*) was caused to perform the verbal action (*duck*). Furthermore, in a spoken sentence, there is an even deeper kind of ambiguity; the first word could have been *eye* or the second word *maid*.

We often introduce the models and algorithms we present throughout the book as ways to **resolve** or **disambiguate** these ambiguities. For example, deciding whether *duck* is a verb or a noun can be solved by **part-of-speech tagging**. Deciding whether *make* means “create” or “cook” can be solved by **word sense disambiguation**. Resolution of part-of-speech and word sense ambiguities are two important kinds of **lexical disambiguation**. A wide variety of tasks can be framed as lexical disambiguation problems. For example, a text-to-speech synthesis system reading the word *lead* needs to decide whether it should be pronounced as in *lead pipe* or as in *lead me on*. By contrast, deciding whether *her* and *duck* are part of the same entity (as in (1.5) or (1.8)) or are different entities (as in (1.6)) is an example of **syntactic disambiguation** and can be addressed by **probabilistic parsing**. We also consider ambiguities that don’t

arise in this particular example, such as determining whether a sentence is a statement or a question (which can be resolved by **speech act interpretation**).

1.3 Models and Algorithms

One of the key insights of the last 50 years of research in language processing is that the various kinds of knowledge described in the last sections can be captured through the use of a small number of formal models or theories. Fortunately, these models and theories are all drawn from the standard toolkits of computer science, mathematics, and linguistics and should be generally familiar to those trained in those fields. Among the most important models are **state machines**, **rule systems**, **logic**, **probabilistic models**, and **vector-space models**. These models, in turn, lend themselves to a small number of algorithms, among the most important of which are **state space search** algorithms, such as **dynamic programming**, and machine learning algorithms, such as **classifiers** and Expectation-Maximization (**EM**) and other learning algorithms.

In their simplest formulation, state machines are formal models that consist of states, transitions among states, and an input representation. Some of the variations of this basic model that we will consider are **deterministic** and **non-deterministic finite-state automata** and **finite-state transducers**.

Closely related to these models are their declarative counterparts: formal rule systems. Among the more important ones we consider (in both probabilistic and non-probabilistic formulations) are **regular grammars** and **regular relations**, **context-free grammars**, and **feature-augmented grammars**. State machines and formal rule systems are the main tools used when dealing with knowledge of phonology, morphology, and syntax.

A third class of models that plays a critical role in capturing knowledge of language are models based on logic. We discuss **first-order logic**, also known as the **predicate calculus**, as well as such related formalisms as lambda-calculus, feature structures, and semantic primitives. These logical representations have traditionally been used for modeling semantics and pragmatics, although more recent work has tended to focus on potentially more robust techniques drawn from non-logical lexical semantics.

Probabilistic models are crucial for capturing every kind of linguistic knowledge. Each of the other models (state machines, formal rule systems, and logic) can be augmented with probabilities. For example, the state machine can be augmented with probabilities to become the **weighted automaton**, or **Markov model**. We spend a significant amount of time on **hidden Markov models** or **HMMs**, which are used everywhere in the field, in part-of-speech tagging, speech recognition, dialogue understanding, text-to-speech, and machine translation. The key advantage of probabilistic models is their ability to solve the many kinds of ambiguity problems that we discussed earlier; almost any speech and language processing problem can be recast as “given N choices for some ambiguous input, choose the most probable one”.

Finally, vector-space models, based on linear algebra, underlie information retrieval and many treatments of word meanings.

Processing language with any of these models typically involves a search through

a space of states representing hypotheses about an input. In speech recognition, we search through a space of phone sequences for the correct word. In parsing, we search through a space of trees for the syntactic parse of an input sentence. In machine translation, we search through a space of translation hypotheses for the correct translation of a sentence into another language. For non-probabilistic tasks, such as tasks involving state machines, we use well-known graph algorithms such as **depth-first search**. For probabilistic tasks, we use heuristic variants such as **best-first** and **A* search** and rely on dynamic programming algorithms for computational tractability.

Machine learning tools such as **classifiers** and **sequence models** play a significant role in many language processing tasks. Based on attributes describing each object, a classifier attempts to assign a single object to a single class while a sequence model attempts to jointly classify a sequence of objects into a sequence of classes.

For example, in the task of deciding whether a word is spelled correctly, classifiers such as **decision trees**, **support vector machines**, **Gaussian mixture models**, and **logistic regression** could be used to make a binary decision (correct or incorrect) for one word at a time. Sequence models such as **hidden Markov models**, **maximum entropy Markov models**, and conditional random fields could be used to assign correct/incorrect labels to all the words in a sentence at once.

Finally, researchers in language processing use many of the same methodological tools that are used in machine learning research—the use of distinct training and test sets, statistical techniques like **cross-validation**, and careful evaluation of trained systems.

1.4 Language, Thought, and Understanding

Turing test

To many, the ability of computers to process language as skillfully as we humans do will signal the arrival of truly intelligent machines. The basis of this belief is the fact that the effective use of language is intertwined with our general cognitive abilities. Among the first to consider the computational implications of this intimate connection was Alan Turing (1950). In this famous paper, Turing introduced what has come to be known as the **Turing test**. Turing began with the thesis that the question of what it would mean for a machine to think was essentially unanswerable because of the inherent imprecision in the terms *machine* and *think*. Instead, he suggested an empirical test, a game, in which a computer's use of language would form the basis for determining if the machine could think. If the machine could win the game, it would be judged intelligent.

In Turing's game, there are three participants: two people and a computer. One of the people is a contestant who plays the role of an interrogator. To win, the interrogator must determine which of the other two participants is the machine by asking a series of questions via a teletype. The task of the machine is to fool the interrogator into believing it is a person by responding as a person would to the interrogator's questions. The task of the second human participant is to convince the interrogator that the other participant is the machine and that she is human.

The following interaction from Turing's paper illustrates the kind of interactions he

had in mind. Clearly, a convincing simulation of humanity doesn't require being an expert in all domains:

Q: Please write me a sonnet on the topic of the Forth Bridge.
 A: Count me out on this one. I never could write poetry.
 Q: Add 34957 to 70764.
 A: (Pause about 30 seconds and then give answer as) 105621.

Turing predicted that roughly by the end of the 20th century a machine with 10 gigabytes of memory would have around a 30% chance of fooling a human interrogator after five minutes of questioning. Given the fact that you can fool some of the people all the time, it is not clear how rigorous this particular standard is. Nevertheless, the critical issue for Turing was that using language as humans do is sufficient, by itself, as an operational test for intelligence.

Surprisingly, many of the issues raised by Turing became relevant as early as 1966 in the context of the **ELIZA** program (Weizenbaum, 1966). ELIZA was an early natural language processing system capable of carrying on a limited form of conversation with a user. Consider the following session with a version of ELIZA that imitated the responses of a Rogerian psychotherapist.

User₁: You are like my father in some ways.
 ELIZA₁: WHAT RESEMBLANCE DO YOU SEE
 User₂: You are not very aggressive but I think you don't want me to notice that.
 ELIZA₂: WHAT MAKES YOU THINK I AM NOT AGGRESSIVE
 User₃: You don't argue with me.
 ELIZA₃: WHY DO YOU THINK I DON'T ARGUE WITH YOU
 User₄: You are afraid of me.
 ELIZA₄: DOES IT PLEASE YOU TO BELIEVE I AM AFRAID OF YOU

As we discuss in more detail in Chapter 2, ELIZA is a remarkably simple program that uses pattern matching to process the input and translate it into suitable outputs. This simple technique succeeds in this domain because ELIZA doesn't actually need to *know* anything to mimic a Rogerian psychotherapist. As Weizenbaum notes, this is one of the few dialogue genres where listeners can act as if they know nothing of the world.

ELIZA's deep relevance to Turing's ideas is that many people who interacted with ELIZA came to believe that it really *understood* them and their problems. Indeed, Weizenbaum (1976) notes that many of these people continued to believe in ELIZA's abilities even after the program's operation was explained to them. In more recent years, Weizenbaum's informal tests have been repeated in a somewhat more controlled setting. Since 1991, an event known as the Loebner Prize competition has attempted to put various computer programs to the Turing test. Although these contests seem to have little scientific interest, a consistent result over the years has been that even the crudest programs can fool some of the judges some of the time (Shieber, 1994a). Not surprisingly, these results have done nothing to quell the ongoing debate over the suitability of the Turing test as a test for intelligence among philosophers and AI researchers (Searle, 1980).

Fortunately, for the purposes of this book, the relevance of these results does not hinge on whether computers will ever be intelligent or will ever understand natural

language. Far more important is recent related research in the social sciences that has confirmed another of Turing's predictions from the same paper.

Nevertheless I believe that at the end of the century the use of words and educated opinion will have altered so much that we will be able to speak of machines thinking without expecting to be contradicted.

It is now clear that regardless of what people believe or know about the inner workings of computers, they talk about them and interact with them as social entities. People act toward computers as if they were people; they are polite to them, treat them as team members, and expect, among other things, that computers should be able to understand their needs and be capable of interacting with them naturally. For example, Reeves and Nass (1996) found that when a computer asked a human to evaluate how well the computer had been doing, the human gives more positive responses than when a different computer asks the same questions. People seemed to be afraid of being impolite. In a different experiment, Reeves and Nass found that people also give computers higher performance ratings if the computer has recently said something flattering to the human. Given these predispositions, speech- and language-based systems may provide many users with the most natural interface for many applications. This fact has led to a long-term focus in the field on the design of **conversational agents**, artificial entities that communicate conversationally.

1.5 The State of the Art

We can only see a short distance ahead, but we can see plenty there that needs to be done.

Alan Turing

This is an exciting time for the field of speech and language processing. The startling increase in computing resources available to the average computer user, the rise of the Web as a massive source of information, and the increasing availability of wireless mobile access have all placed speech- and language-processing applications in the technology spotlight. The following are examples of some currently deployed systems that reflect this trend:

- Travelers calling Amtrak, United Airlines, and other travel providers interact with conversational agents that guide them through the process of making reservations and getting arrival and departure information.
- Car makers provide automatic speech recognition and text-to-speech systems that allow drivers to control their environmental, entertainment, and navigational systems by voice. A similar spoken dialogue system has been deployed by astronauts on the International Space Station.
- Video search companies provide search services for millions of hours of video on the Web by using speech recognition technology to capture the words in the sound track.
- Google provides cross-language information retrieval and translation services whereby users can supply queries in their native language to search collections

in another language. Google translates the query, finds the most relevant pages, and then automatically translates them back to the user's native language.

- Large educational publishers such as Pearson and testing services like ETS use automated systems to analyze thousands of student essays, grading and assessing them in a manner that is indistinguishable from human graders.
- Interactive virtual agents, based on lifelike animated characters, serve as tutors for children learning to read (Wise et al., 2007).
- Text analysis companies provide marketing intelligence based on automated measurements of user opinions, preferences, attitudes as expressed in weblogs, discussion forums, and user groups.

1.6 Some Brief History

Historically, speech and language processing has been treated very differently in computer science, electrical engineering, linguistics, and psychology/cognitive science. Because of this diversity, speech and language processing encompasses a number of different but overlapping fields in these different departments: **computational linguistics** in linguistics, **natural language processing** in computer science, **speech recognition** in electrical engineering, **computational psycholinguistics** in psychology. This section summarizes the different historical threads that have given rise to the field of speech and language processing. This section provides only a sketch, but many of the topics listed here are covered in more detail in subsequent chapters.

1.6.1 Foundational Insights: 1940s and 1950s

The earliest roots of the field date to the intellectually fertile period just after World War II that gave rise to the computer itself. This period from the 1940s through the end of the 1950s saw intense work on two foundational paradigms: the **automaton** and **probabilistic** or **information-theoretic models**.

The automaton arose in the 1950s out of Turing's (1936) model of algorithmic computation, considered by many to be the foundation of modern computer science. Turing's work led first to the **McCulloch-Pitts neuron** (McCulloch and Pitts, 1943), a simplified model of the neuron as a kind of computing element that could be described in terms of propositional logic, and then to the work of Kleene (1951) and (1956) on finite automata and regular expressions. Shannon (1948) applied probabilistic models of discrete Markov processes to automata for language. Drawing on the idea of a finite-state Markov process from Shannon's work, Chomsky (1956) first considered finite-state machines as a way to characterize a grammar and defined a finite-state language as a language generated by a finite-state grammar. These early models led to the field of **formal language theory**, which used algebra and set theory to define formal languages as sequences of symbols. This includes the context-free grammar, first defined by Chomsky (1956) for natural languages but independently discovered by Backus (1959) and Naur et al. (1960) in their descriptions of the ALGOL programming language.

The second foundational insight of this period was the development of probabilistic algorithms for speech and language processing, which dates to Shannon's other contribution: the metaphor of the **noisy channel** and **decoding** for the transmission of language through media such as communication channels and speech acoustics. Shannon also borrowed the concept of **entropy** from thermodynamics as a way of measuring the information capacity of a channel, or the information content of a language, and performed the first measure of the entropy of English by using probabilistic techniques.

It was also during this early period that the sound spectrograph was developed (Koenig et al., 1946), and foundational research was done in instrumental phonetics that laid the groundwork for later work in speech recognition. This led to the first machine speech recognizers in the early 1950s. In 1952, researchers at Bell Labs built a statistical system that could recognize any of the 10 digits from a single speaker (Davis et al., 1952). The system had 10 speaker-dependent stored patterns roughly representing the first two vowel formants in the digits. They achieved 97%–99% accuracy by choosing the pattern that had the highest relative correlation coefficient with the input.

1.6.2 The Two Camps: 1957–1970

By the end of the 1950s and the early 1960s, speech and language processing had split very cleanly into two paradigms: symbolic and stochastic.

The symbolic paradigm took off from two lines of research. The first was the work of Chomsky and others on formal language theory and generative syntax throughout the late 1950s and early to mid 1960s, and the work of many linguistics and computer scientists on parsing algorithms, initially top-down and bottom-up and then with dynamic programming. One of the earliest complete parsing systems was Zelig Harris's Transformations and Discourse Analysis Project (TDAP), which was implemented between June 1958 and July 1959 at the University of Pennsylvania (Harris, 1962).² The second line of research was the new field of artificial intelligence. In the summer of 1956 John McCarthy, Marvin Minsky, Claude Shannon, and Nathaniel Rochester brought together a group of researchers for a two-month workshop on what they decided to call artificial intelligence (AI). Although AI always included a minority of researchers focusing on stochastic and statistical algorithms (including probabilistic models and neural nets), the major focus of the new field was the work on reasoning and logic typified by Newell and Simon's work on the Logic Theorist and the General Problem Solver. At this point, early natural language understanding systems were built. These simple systems worked in single domains mainly by a combination of pattern matching and keyword search with simple heuristics for reasoning and question-answering. By the late 1960s, more formal logical systems were developed.

The stochastic paradigm took hold mainly in departments of statistics and of electrical engineering. By the late 1950s, the Bayesian method was beginning to be applied to the problem of optical character recognition. Bledsoe and Browning (1959) built a Bayesian text-recognition that used a large dictionary and computed the likelihood of each observed letter sequence given each word in the dictionary by multiplying the

² This system was reimplemented recently and is described by Joshi and Hopely (1999) and Karttunen (1999), who note that the parser was essentially implemented as a cascade of finite-state transducers.

likelihoods for each letter. Mosteller and Wallace (1964) applied Bayesian methods to the problem of authorship attribution on *The Federalist* papers.

The 1960s also saw the rise of the first serious testable psychological models of human language processing based on transformational grammar, as well as the first on-line corpora: the Brown corpus of American English, a one-million-word collection of samples from 500 written texts from different genres (newspaper, novels, non-fiction, academic, etc.), which was assembled at Brown University in 1963–64 (Kučera and Francis, 1967; Francis, 1979; Francis and Kučera, 1982), and William S. Y. Wang’s 1967 DOC (Dictionary on Computer), an on-line Chinese dialect dictionary.

1.6.3 Four Paradigms: 1970–1983

The next period saw an explosion in research in speech and language processing and the development of a number of research paradigms that still dominate the field.

The **stochastic** paradigm played a huge role in the development of speech recognition algorithms in this period, particularly the use of the hidden Markov model (HMM) and the metaphors of the noisy channel and decoding, developed independently by Jelinek, Bahl, Mercer, and colleagues at IBM’s Thomas J. Watson Research Center, and by Baker at Carnegie Mellon University, who was influenced by the work of Baum and colleagues at the Institute for Defense Analyses in Princeton. AT&T’s Bell Laboratories was another key center for work on speech recognition and synthesis; see Rabiner and Juang (1993) for descriptions of the wide range of this work.

The **logic-based** paradigm was begun by the work of Colmerauer and his colleagues on Q-systems and metamorphosis grammars (Colmerauer, 1970, 1975), the forerunners of Prolog, and Definite Clause Grammars (Pereira and Warren, 1980). Independently, Kay’s (1979) work on functional grammar and shortly later, Bresnan and Kaplan’s (1982) work on Lexical Functional Grammar (LFG), established the importance of feature structure unification.

The **natural language understanding** field took off during this period, beginning with Winograd’s SHRDLU system, which simulated a robot embedded in a world of toy blocks (Winograd, 1972a). The program was able to accept natural-language text commands (*Move the red block on top of the smaller green one*) of a hitherto unseen complexity and sophistication. His system was also the first to attempt to build an extensive (for the time) grammar of English, based on Halliday’s systemic grammar. Winograd’s model made it clear that the problem of parsing was well enough understood to begin to focus on semantics and discourse. Roger Schank and his colleagues and students (in what was often referred to as the *Yale School*) built a series of language-understanding programs that focused on conceptual knowledge such as scripts, plans, and goals, and human memory organization (Schank and Abelson, 1977; Schank and Riesbeck, 1981; Cullingford, 1981; Wilensky, 1983; Lehnert, 1977). This work often used network-based semantics (Quillian, 1968; Norman and Rumelhart, 1975; Schank, 1972; Wilks, 1975c, 1975b; Kintsch, 1974) and began to incorporate Fillmore’s notion of case roles (Fillmore, 1968) into their representations (Simmons, 1973).

The logic-based and natural-language-understanding paradigms were unified in systems that used predicate logic as a semantic representation, such as the LUNAR question-answering system (Woods, 1967, 1973).

The **discourse modeling** paradigm focused on four key areas in discourse. Grosz and her colleagues introduced the study of substructure in discourse, and of discourse focus (Grosz, 1977a; Sidner, 1983); a number of researchers began to work on automatic reference resolution (Hobbs, 1978); and the **BDI** (Belief-Desire-Intention) framework for logic-based work on speech acts was developed (Perrault and Allen, 1980; Cohen and Perrault, 1979).

1.6.4 Empiricism and Finite-State Models Redux: 1983–1993

This next decade saw the return of two classes of models that had lost popularity in the late 1950s and early 1960s, partially due to theoretical arguments against them such as Chomsky’s influential review of Skinner’s *Verbal Behavior* (Chomsky, 1959b). The first class was finite-state models, which began to receive attention again after work on finite-state phonology and morphology by Kaplan and Kay (1981) and finite-state models of syntax by Church (1980). A large body of work on finite-state models is described throughout the book.

The second trend in this period was what has been called the “return of empiricism”; most notable here was the rise of probabilistic models throughout speech and language processing, influenced strongly by the work at the IBM Thomas J. Watson Research Center on probabilistic models of speech recognition. These probabilistic methods and other such data-driven approaches spread from speech into part-of-speech tagging, parsing and attachment ambiguities, and semantics. This empirical direction was also accompanied by a new focus on model evaluation, based on using held-out data, developing quantitative metrics for evaluation, and emphasizing the comparison of performance on these metrics with previous published research.

This period also saw considerable work on natural language generation.

1.6.5 The Field Comes Together: 1994–1999

By the last five years of the millennium it was clear that the field was undergoing major changes. First, probabilistic and data-driven models had become quite standard throughout natural language processing. Algorithms for parsing, part-of-speech tagging, reference resolution, and discourse processing all began to incorporate probabilities and to employ evaluation methodologies borrowed from speech recognition and information retrieval. Second, the increases in the speed and memory of computers had allowed commercial exploitation of a number of subareas of speech and language processing, in particular, speech recognition, and spelling and grammar correction. Speech and language processing algorithms began to be applied to Augmentative and Alternative Communication (AAC). Finally, the rise of the Web emphasized the need for language-based information retrieval and information extraction.

1.6.6 The Rise of Machine Learning: 2000–2008

The empiricist trends begun in the latter part of the 1990s accelerated at an astounding pace in the new century. This acceleration was largely driven by three synergistic trends.

First, large amounts of spoken and written material became widely available through the auspices of the Linguistic Data Consortium (LDC) and other similar organizations. Importantly, included among these materials were annotated collections such as the Penn Treebank (Marcus et al., 1993), Prague Dependency Treebank (Hajič, 1998), PropBank (Palmer et al., 2005), Penn Discourse Treebank (Miltsakaki et al., 2004b), RSTBank (Carlson et al., 2001) and TimeBank (Pustejovsky et al., 2003b), all of which layered standard text sources with various forms of syntactic, semantic, and pragmatic annotations. The existence of these resources promoted the trend of casting more complex traditional problems, such as parsing and semantic analysis, as problems in supervised machine learning. These resources also promoted the establishment of additional competitive evaluations for parsing (Dejean and Tjong Kim Sang, 2001), information extraction (NIST, 2007a; Tjong Kim Sang, 2002; Tjong Kim Sang and De Meulder, 2003), word sense disambiguation (Palmer et al., 2001; Kilgariff and Palmer, 2000), question answering (Voorhees and Tice, 1999), and summarization (Dang, 2006).

Second, this increased focus on learning led to a more serious interplay with the statistical machine learning community. Techniques such as support vector machines (Boser et al., 1992; Vapnik, 1995), maximum entropy techniques and their equivalent formulation as multinomial logistic regression (Berger et al., 1996), and graphical Bayesian models (Pearl, 1988) became standard practice in computational linguistics.

Third, the widespread availability of high-performance computing systems facilitated the training and deployment of systems that could not have been imagined a decade earlier.

Finally, near the end of this period, largely unsupervised statistical approaches began to receive renewed attention. Progress on statistical approaches to machine translation (Brown et al., 1990; Och and Ney, 2003) and topic modeling (Blei et al., 2003) demonstrated that effective applications could be constructed from systems trained on unannotated data alone. In addition, the widespread cost and difficulty of producing reliably annotated corpora became a limiting factor in the use of supervised approaches for many problems. This trend toward the use of unsupervised techniques will likely increase.

1.6.7 On Multiple Discoveries

Even in this brief historical overview, we have mentioned a number of cases of multiple independent discoveries of the same idea. Just a few of the “multiples” to be discussed in this book include the application of dynamic programming to sequence comparison by Viterbi, Vintsyuk, Needleman and Wunsch, Sakoe and Chiba, Sankoff, Reichert et al., and Wagner and Fischer (Chapters 3, 5, and 6); the HMM/noisy channel model of speech recognition by Baker and by Jelinek, Bahl, and Mercer (Chapters 6, 9, and 10); the development of context-free grammars by Chomsky and by Backus and Naur (Chapter 12); the proof that Swiss-German has a non-context-free syntax by Huybregts and by Shieber (Chapter 16); the application of unification to language processing by Colmerauer et al. and by Kay (Chapter 15).

Are these multiples to be considered astonishing coincidences? A well-known hypothesis by sociologist of science Robert K. Merton (1961) argues, quite the contrary, that

all scientific discoveries are in principle multiples, including those that on the surface appear to be singletons.

Of course, there are many well-known cases of multiple discovery or invention; just a few examples from an extensive list in Ogburn and Thomas (1922) include the multiple invention of the calculus by Leibnitz and by Newton, the multiple development of the theory of natural selection by Wallace and by Darwin, and the multiple invention of the telephone by Gray and Bell.³ But Merton gives a further array of evidence for the hypothesis that multiple discovery is the rule rather than the exception, including many cases of putative singletons that turn out to be a rediscovery of previously unpublished or perhaps inaccessible work. An even stronger piece of evidence is his ethnomethodological point that scientists themselves act under the assumption that multiple invention is the norm. Thus many aspects of scientific life are designed to help scientists avoid being “scooped”: submission dates on journal articles, careful dates in research records, circulation of preliminary or technical reports.

1.6.8 A Final Brief Note on Psychology

Many of the chapters in this book include short summaries of psychological research on human processing. Of course, understanding human language processing is an important scientific goal in its own right and is part of the general field of cognitive science. However, an understanding of human language processing can often be helpful in building better machine models of language. This seems contrary to the popular wisdom, which holds that direct mimicry of nature’s algorithms is rarely useful in engineering applications. For example, the argument is often made that if we copied nature exactly, airplanes would flap their wings; yet airplanes with fixed wings are a more successful engineering solution. But language is not aeronautics. Cribbing from nature is sometimes useful for aeronautics (after all, airplanes do have wings), but it is particularly useful when we are trying to solve human-centered tasks. Airplane flight has different goals from bird flight; but the goal of speech recognition systems, for example, is to perform exactly the task that human court reporters perform every day: transcribe spoken dialog. Since people already do this well, we can learn from nature’s previous solution. Furthermore, since an important application of speech and language processing systems is for human-computer interaction, it makes sense to copy a solution that behaves the way people are accustomed to.

1.7 Summary

This chapter introduces the field of speech and language processing. The following are some of the highlights of this chapter.

³ Ogburn and Thomas are generally credited with noticing that the prevalence of multiple inventions suggests that the cultural milieu and not individual genius is the deciding causal factor in scientific discovery. In an amusing bit of recursion, however, Merton notes that even this idea has been multiply discovered, citing sources from the 19th century and earlier!

- A good way to understand the concerns of speech and language processing research is to consider what it would take to create an intelligent agent like HAL from *2001: A Space Odyssey*, or build a Web-based question answerer, or a machine translation engine.
- Speech and language technology relies on formal models, or representations, of knowledge of language at the levels of phonology and phonetics, morphology, syntax, semantics, pragmatics and discourse. A number of formal models including state machines, formal rule systems, logic, and probabilistic models are used to capture this knowledge.
- The foundations of speech and language technology lie in computer science, linguistics, mathematics, electrical engineering, and psychology. A small number of algorithms from standard frameworks are used throughout speech and language processing.
- The critical connection between language and thought has placed speech and language processing technology at the center of debate over intelligent machines. Furthermore, research on how people interact with complex media indicates that speech and language processing technology will be critical in the development of future technologies.
- Revolutionary applications of speech and language processing are currently in use around the world. The creation of the Web, as well as significant recent improvements in speech recognition and synthesis, will lead to many more applications.

Bibliographical and Historical Notes

Research in the various subareas of speech and language processing is spread across a wide number of conference proceedings and journals. The conferences and journals most centrally concerned with natural language processing and computational linguistics are associated with the Association for Computational Linguistics (ACL), its European counterpart (EACL), and the International Conference on Computational Linguistics (COLING). The annual proceedings of ACL, NAACL, and EACL, and the biennial COLING conference are the primary forums for work in this area. Related conferences include various proceedings of ACL Special Interest Groups (SIGs) such as the Conference on Natural Language Learning (CoNLL), as well as the conference on Empirical Methods in Natural Language Processing (EMNLP).

Research on speech recognition, understanding, and synthesis is presented at the annual INTERSPEECH conference, which is called the International Conference on Spoken Language Processing (ICSLP), and the European Conference on Speech Communication and Technology (EUROSPEECH) in alternating years, or the annual IEEE International Conference on Acoustics, Speech, and Signal Processing (IEEE ICASSP). Spoken language dialogue research is presented at these or at workshops like SIGDial.

Journals include *Computational Linguistics*, *Natural Language Engineering*, *Computer Speech and Language*, *Speech Communication*, the *IEEE Transactions on Audio*,

Speech & Language Processing, the *ACM Transactions on Speech and Language Processing*, and *Linguistic Issues in Language Technology*.

Many of these papers, including those from the *Computational Linguistics* journal and the *ACL*, *COLING*, and related conferences are available for free online at the *ACL Anthology* (<http://www.aclweb.org/anthology-index/>).

Work on language processing from an Artificial Intelligence perspective can be found in the annual meetings of the American Association for Artificial Intelligence (AAAI), as well as the biennial International Joint Conference on Artificial Intelligence (IJCAI) meetings. Artificial intelligence journals that periodically feature work on speech and language processing include *Machine Learning*, *Journal of Machine Learning Research*, and the *Journal of Artificial Intelligence Research*.

A fair number of textbooks covering various aspects of speech and language processing are available. Manning and Schütze (1999) (*Foundations of Statistical Language Processing*) focuses on statistical models of tagging, parsing, disambiguation, collocations, and other areas. Charniak (1993) (*Statistical Language Learning*) is an accessible, though older and less-extensive, introduction to similar material. Manning et al. (2008) focuses on information retrieval, text classification, and clustering. NLTK, the Natural Language Toolkit (Bird and Loper, 2004), is a suite of Python modules and data for natural language processing, together with a Natural Language Processing book based on the NLTK suite. Allen (1995) (*Natural Language Understanding*) provides extensive coverage of language processing from the AI perspective. Gazdar and Mellish (1989) (*Natural Language Processing in Lisp/Prolog/Pop11*) covers especially automata, parsing, features, and unification and is available free online. Pereira and Shieber (1987) gives a Prolog-based introduction to parsing and interpretation. Russell and Norvig (2002) introduce artificial intelligence with chapters on natural language processing. Partee et al. (1990) broadly cover mathematical linguistics. A historically significant collection of foundational papers can be found in Grosz et al. (1986) (*Readings in Natural Language Processing*).

There are many places to acquire corpora of speech and text. One of the largest is the Linguistic Data Consortium, a non-profit consortium that creates and distributes corpora (<http://www ldc.upenn.edu/>). See the textbook web page for pointers to others such as CHILDES (<http://childes.psy.cmu.edu/>), the British National Corpus (<http://www.natcorp.ox.ac.uk/>), the International Corpus of English (<http://www.ucl.ac.uk/english-usage/ice/index.htm>), and Project Gutenberg (<http://www.gutenberg.org/>).